

Module : Programmation Orientée Objet (POO)

Domaine: MI

Dr. M. A. BOUZAR

Filière: Informatique

Plan du cours

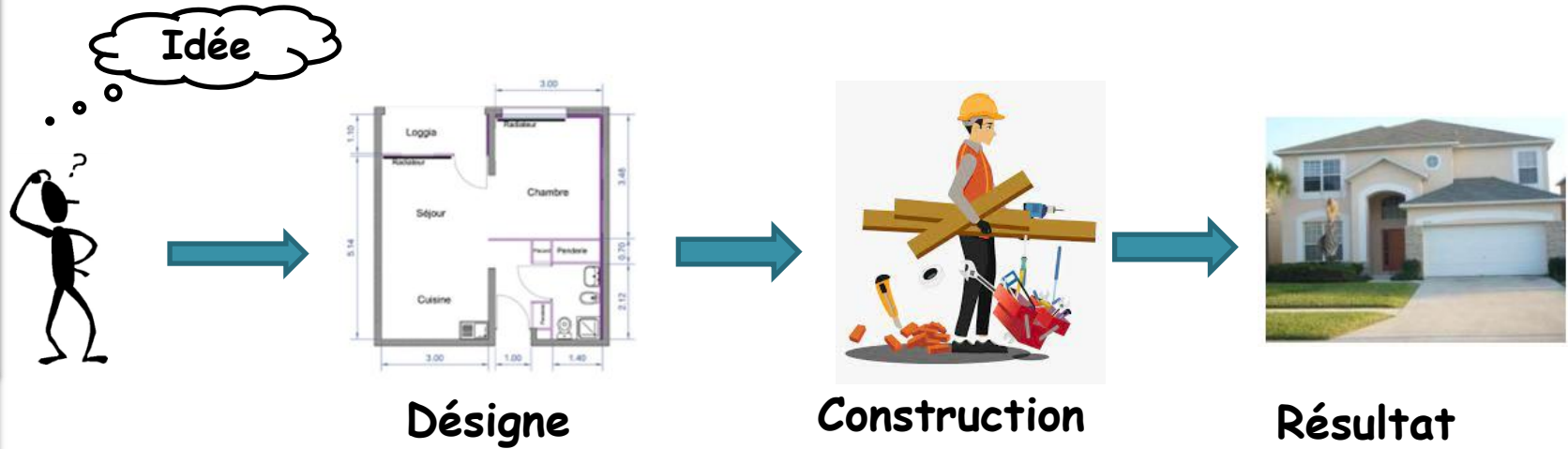
- 1) Introduction à la programmation **orientée objet**
- 2) Initiation à la programmation en **JAVA**
- 3) La programmation **orientée objet** avec **JAVA**
 - **Classes et objets**
 - **Héritage, Polymorphisme et abstraction**
 - **Classes abstraites et Interfaces**



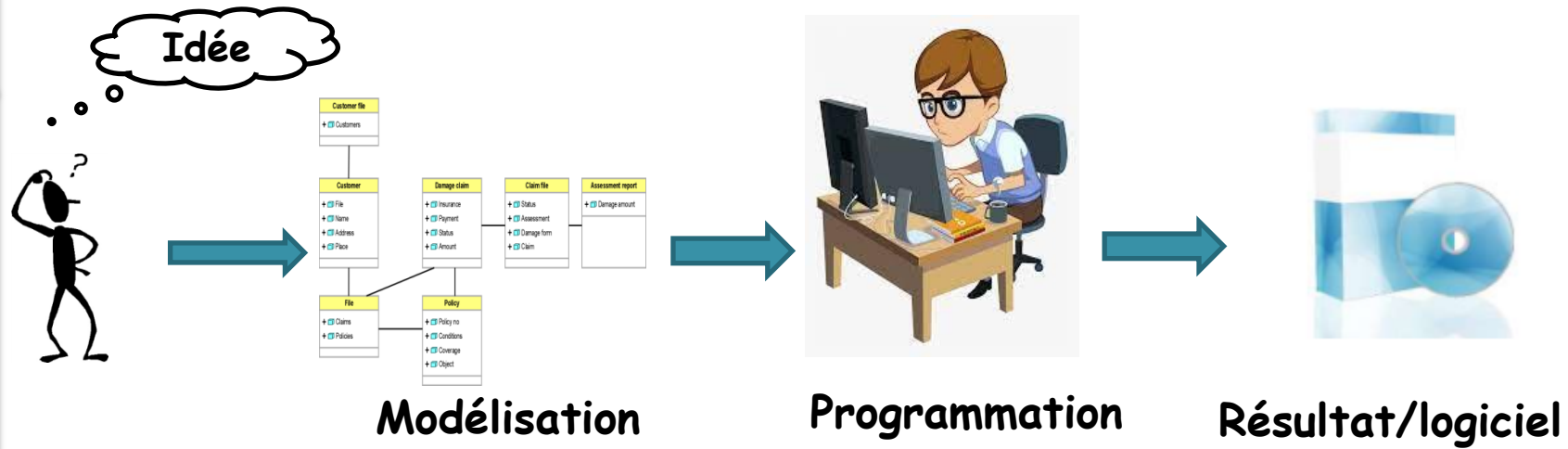
Introduction



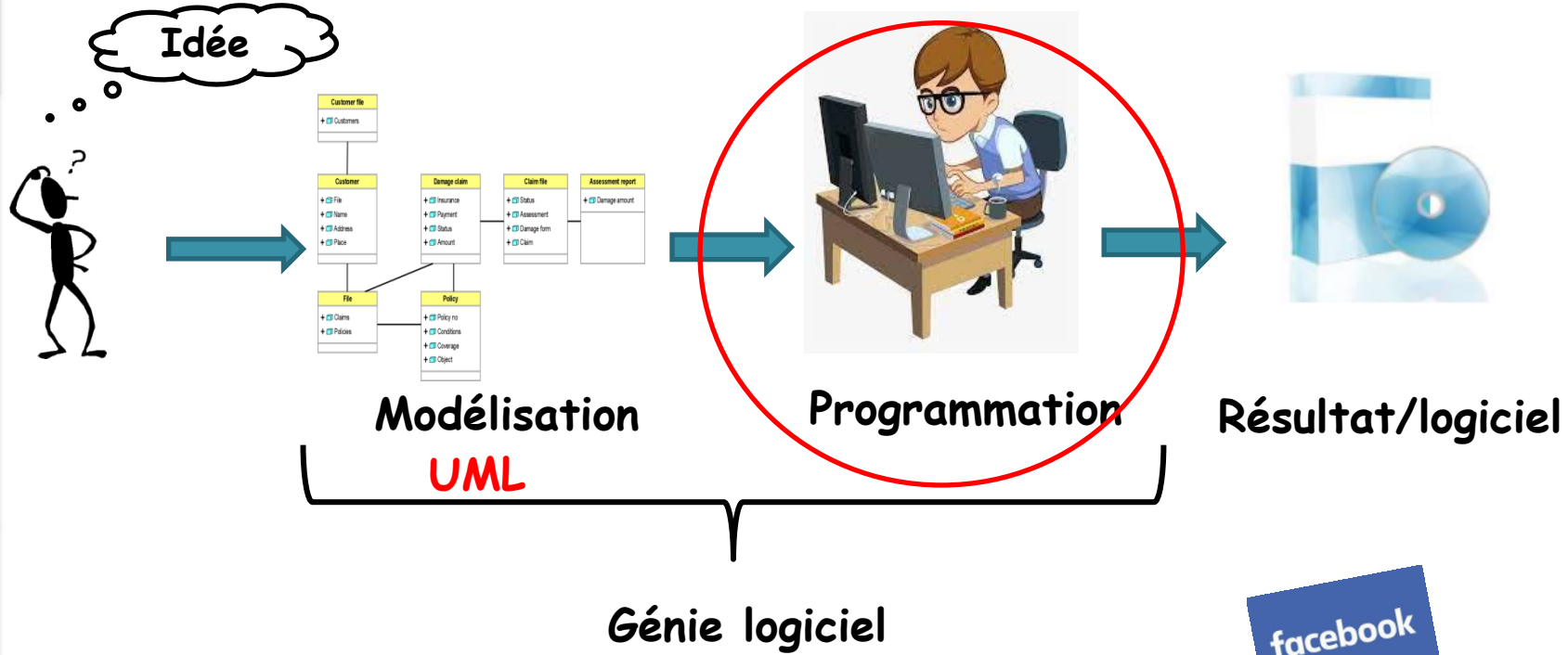
Introduction



Introduction JAVA

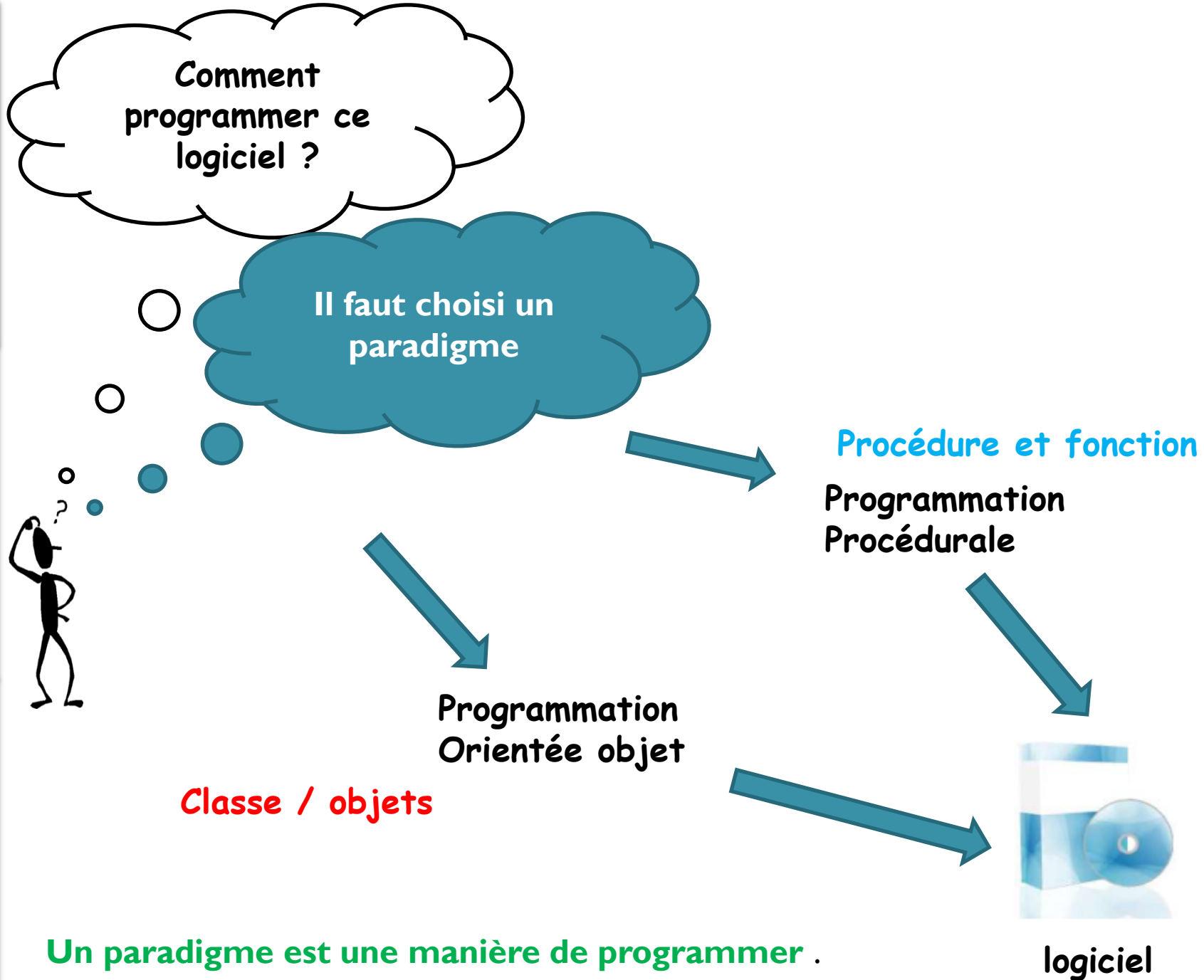


OOP avec JAVA



facebook

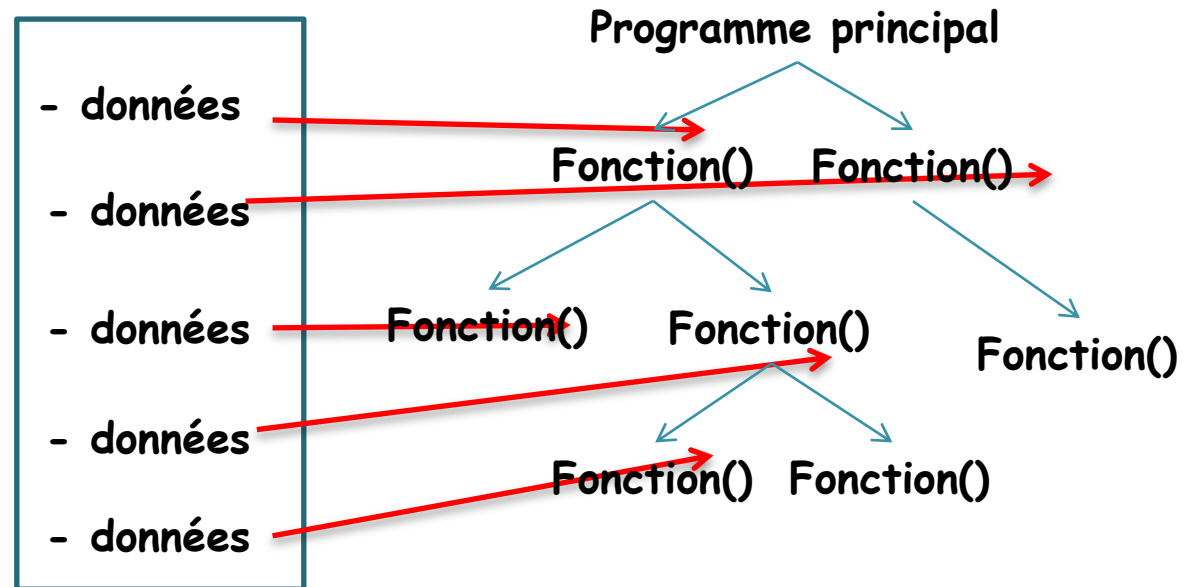
Google





logiciel

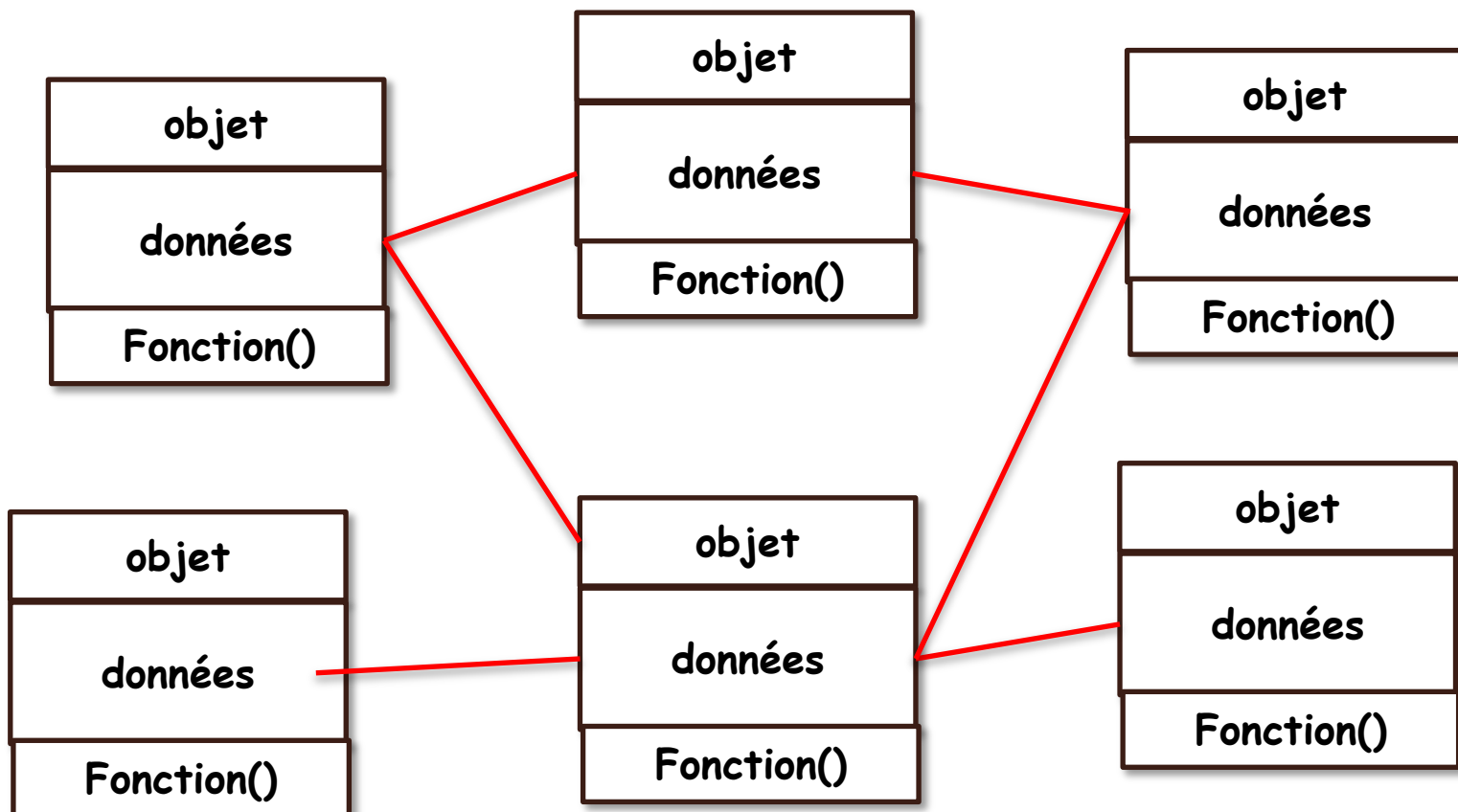
Programmation
Procédurale





logiciel

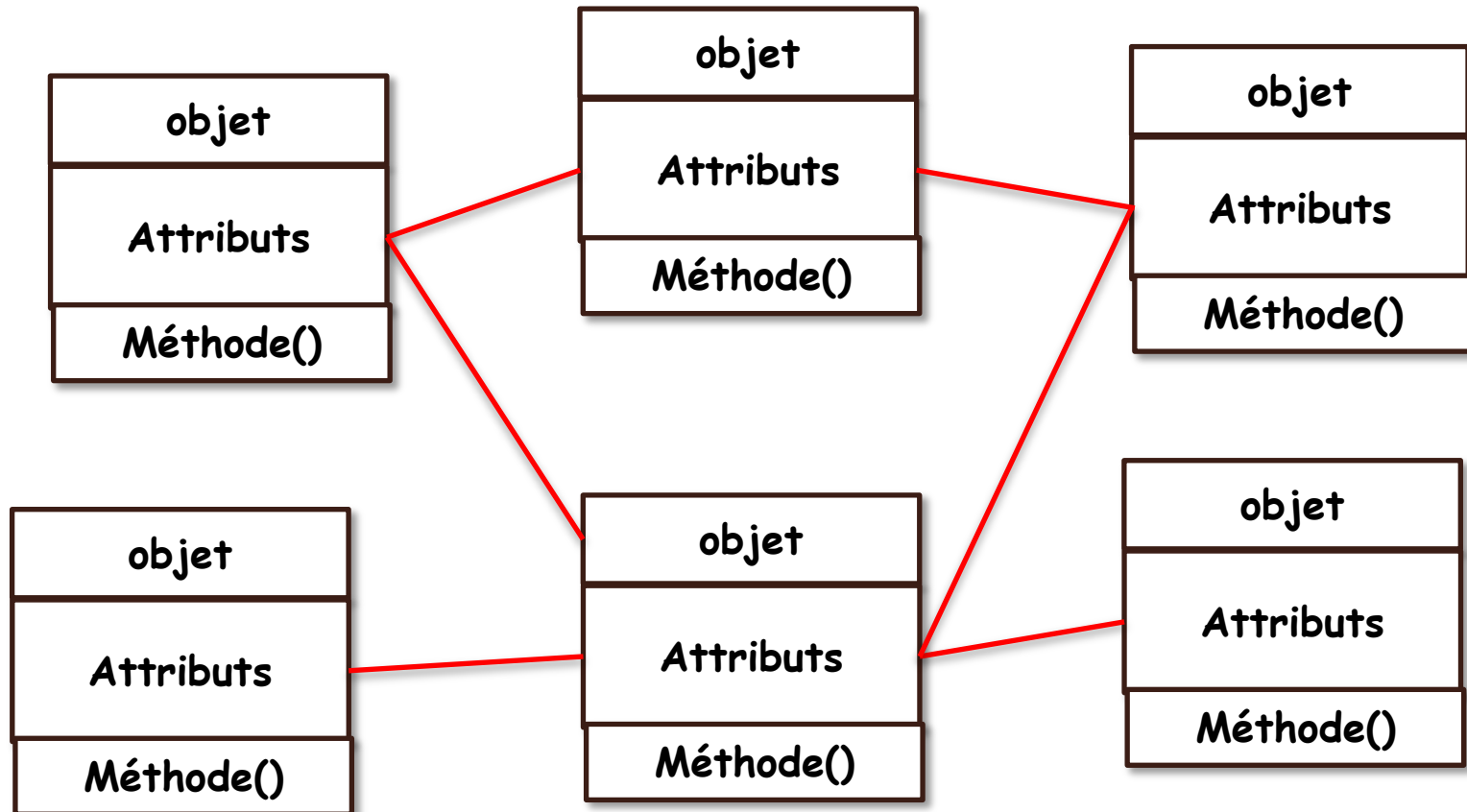
Programmation
Orientée objet





logiciel

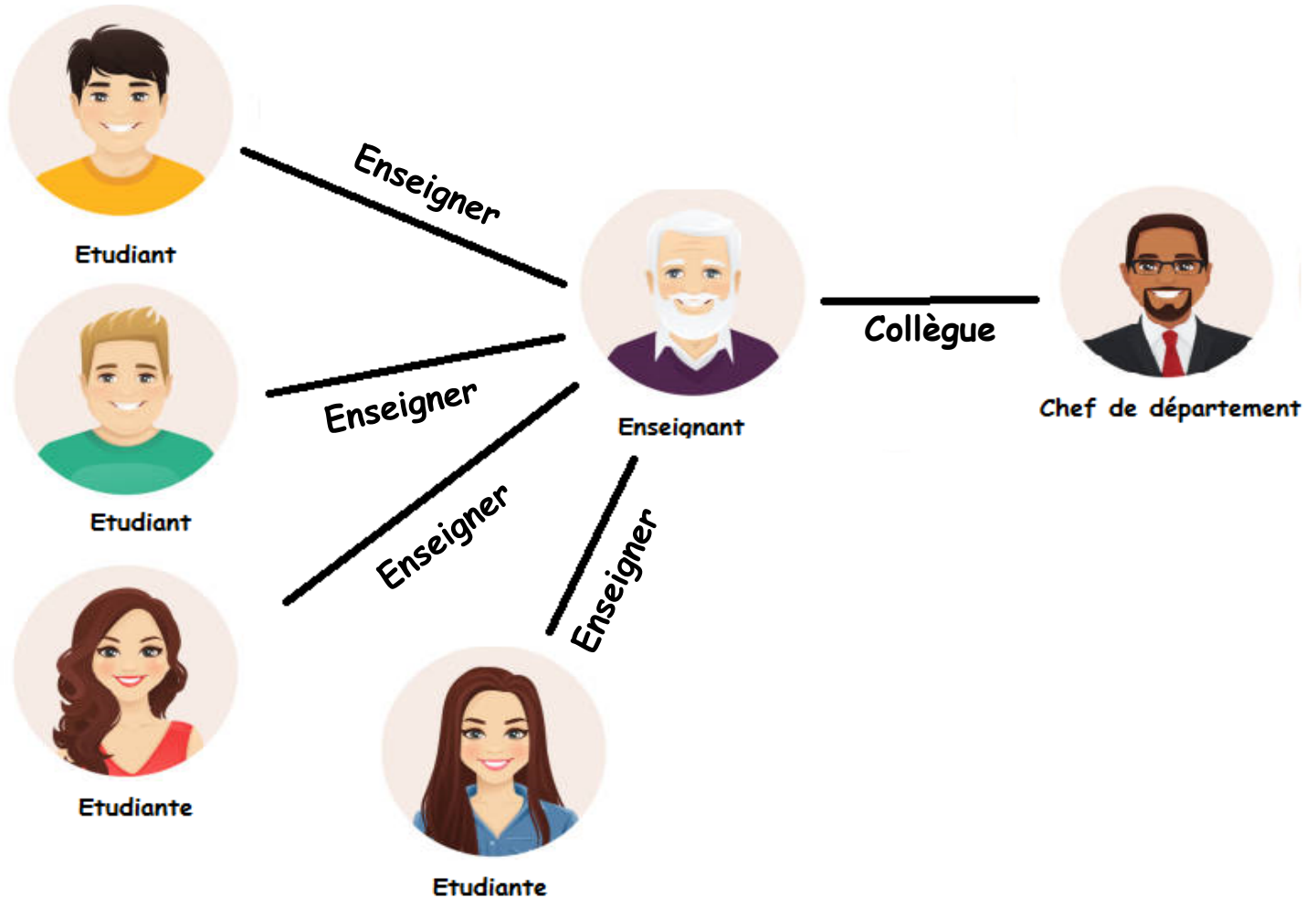
Programmation
Orientée objet





logiciel

Programmation
Orientée objet



Introduction

Introduction JAVA

OOP avec JAVA



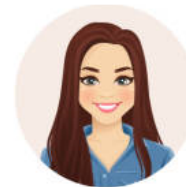
Etudiant
Nom: Richard Prénom: Simon Age: 24 Sexe: M
Etudier() Transférer()



Etudiant
Nom: Thomas Prénom: Louis Age: 21 Sexe: M
Etudier() Transférer()



Etudiante
Nom: Emma Prénom: Vincent Age: 23 Sexe: F
Etudier() Transférer()



Etudiante
Nom: Alice Prénom: Klein Age: 22 Sexe: F
Etudier() Transférer()



Enseignant
Nom: Bernard Prénom: Durand Age: 57 Sexe: F
Démissionner() Enseigner() Mutation()

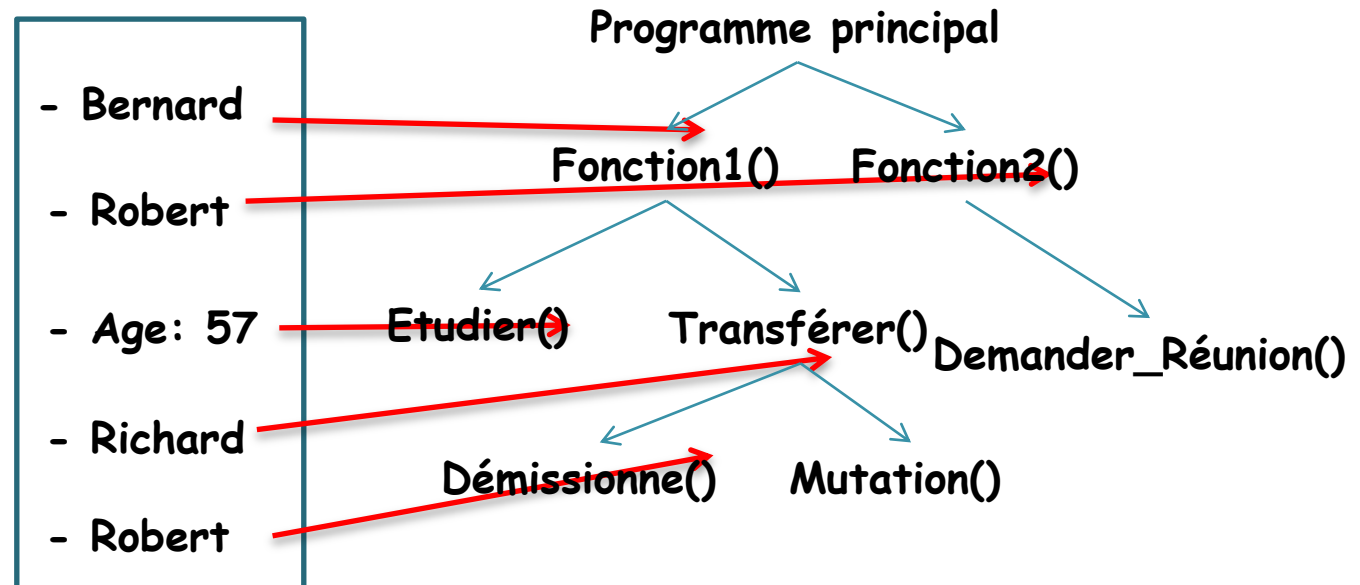


Chef de département
Nom: Robert Prénom: Dubois Age: 42 Sexe: M
Démissionner() Mutation() Demander_Réunion()

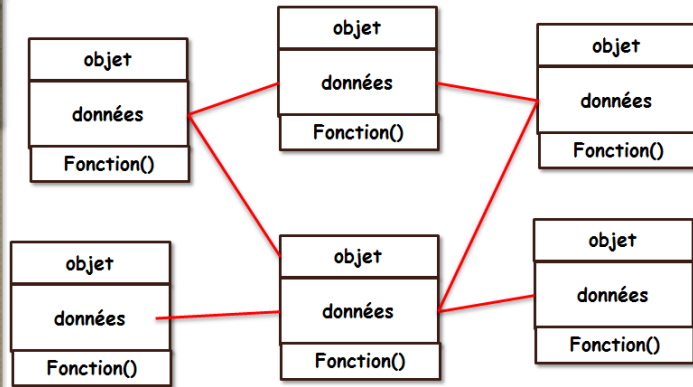


logiciel

Programmation
Procédurale

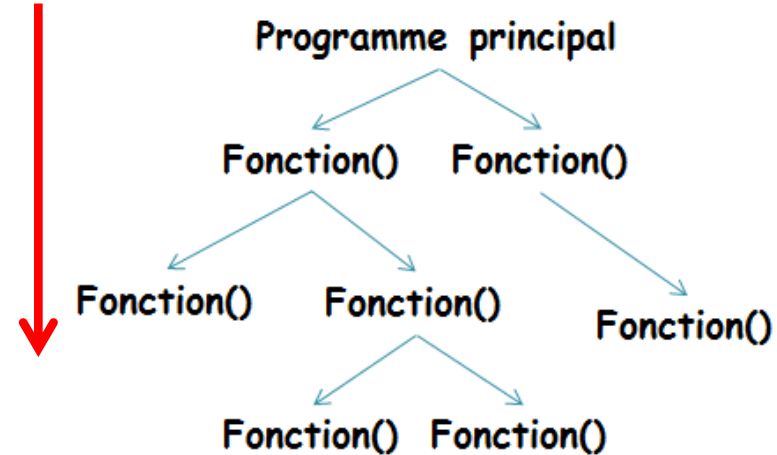


Programmation Orientée objet



- **Divisé en objets**
- **A des règles d'accès**
- **Les données sont liées à l'objet**
Chaque objet peut se déplacer à travers ses méthodes
- **Plus sécurisé**

Programmation Procédural



- **Divisé en fonctions**
- **N'a pas de règles d'accès**
- **Les données peuvent circuler librement d'une fonction à l'autre**
- **Moins sécurisé**

OOP VS procédural

Langage de Programmation Orientée objet

- **Simula (1967)**
- **Smalltalk (les années 70)**
- **Object PASCAL (Delphi)**
- **C++**
- **C#**
- **PHP**
- **JAVA**

Langage de Programmation Procédural

- **PASCAL**
- **Basic**
- **C**
- **C++**

Les raisons du succès de la POO

- Fondée sur une solide approche **génie logiciel** ;
- Maîtrise de la complexité de grands systèmes logiciels ;
- Consolidation par la **modélisation orientée objet** et le standard UML.

Langage Haut niveau



Pour humains

Code source

Byte Code



Langage Bas niveau

Pour la machine

machine
CodeExtension ***.java**

```
Public static void main () {  
    System.out.println("Bonjour");  
}
```

IDE: Integrated Development Environment (Code Editor, Compiler, Debugger)

JDK: Java development Kit

Extension ***.Class (portable)**

JRE: Java Runtime Environment

JVM: Java Virtual machine

0100101110100010111101001001

Comment débiter avec java ?

1 Installation de JDK:

www.oracle.com

2 Installation de IDE:

Eclipse IDE: www.eclipse.org

Netbeans IDE: www.netbeans.apache.org

Intellij : www.jetbrains.com

Conventions de nommage

- Organisation des fichiers
- Le fichier **.java** doit avoir le même nom que la classe publique qu'il décrit

Class Etudiant  Etudiant.**java**
- Le fichier **.java** par classe, même pour celle contenant le **main()**

Conventions de nommage

- Éléments de base:

paquetages minuscules

Classes: MajusculePourLaPremiereLettre
DeChaqueMot

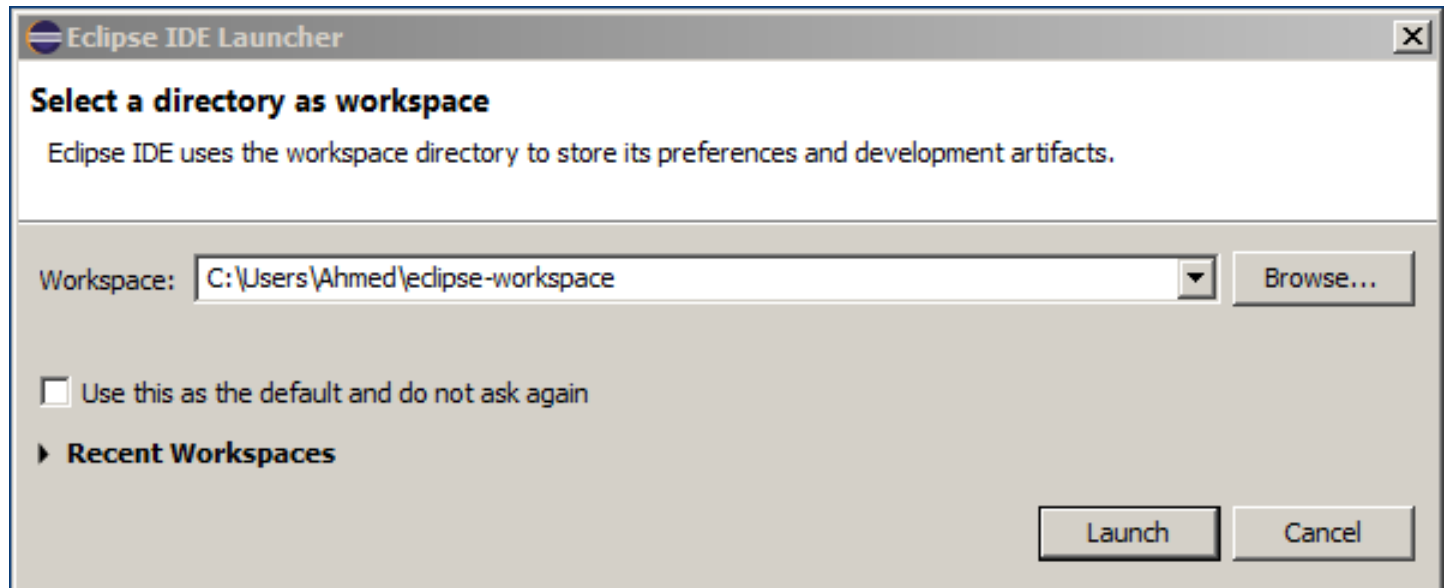
Interface: MajusculePourLaPremiereLettre
DeChaqueMot

méthodes minusculePourLaPremiereLettre

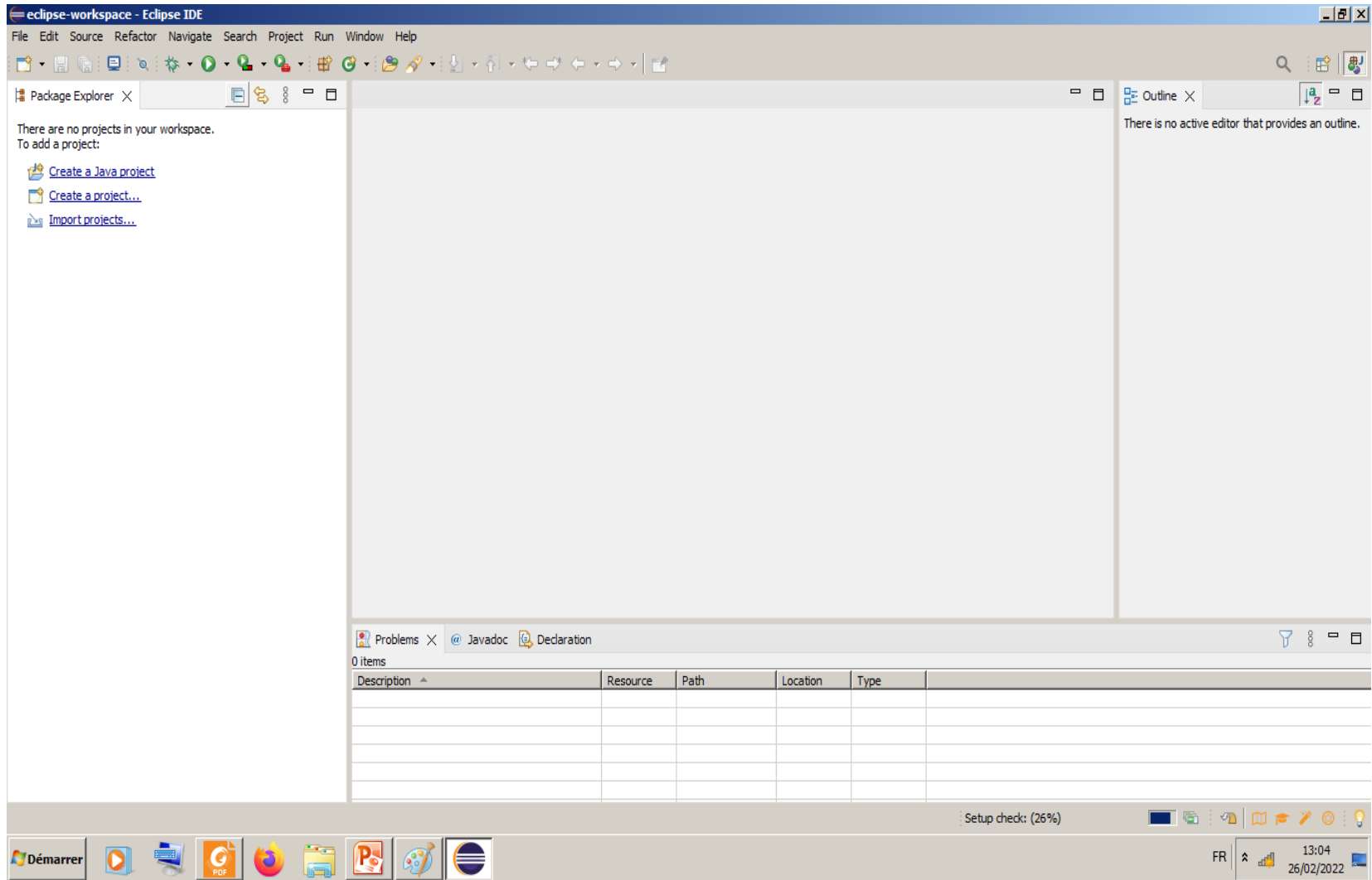
constantes MAJUSCULES

variables minusculePourLaPremiereLettre

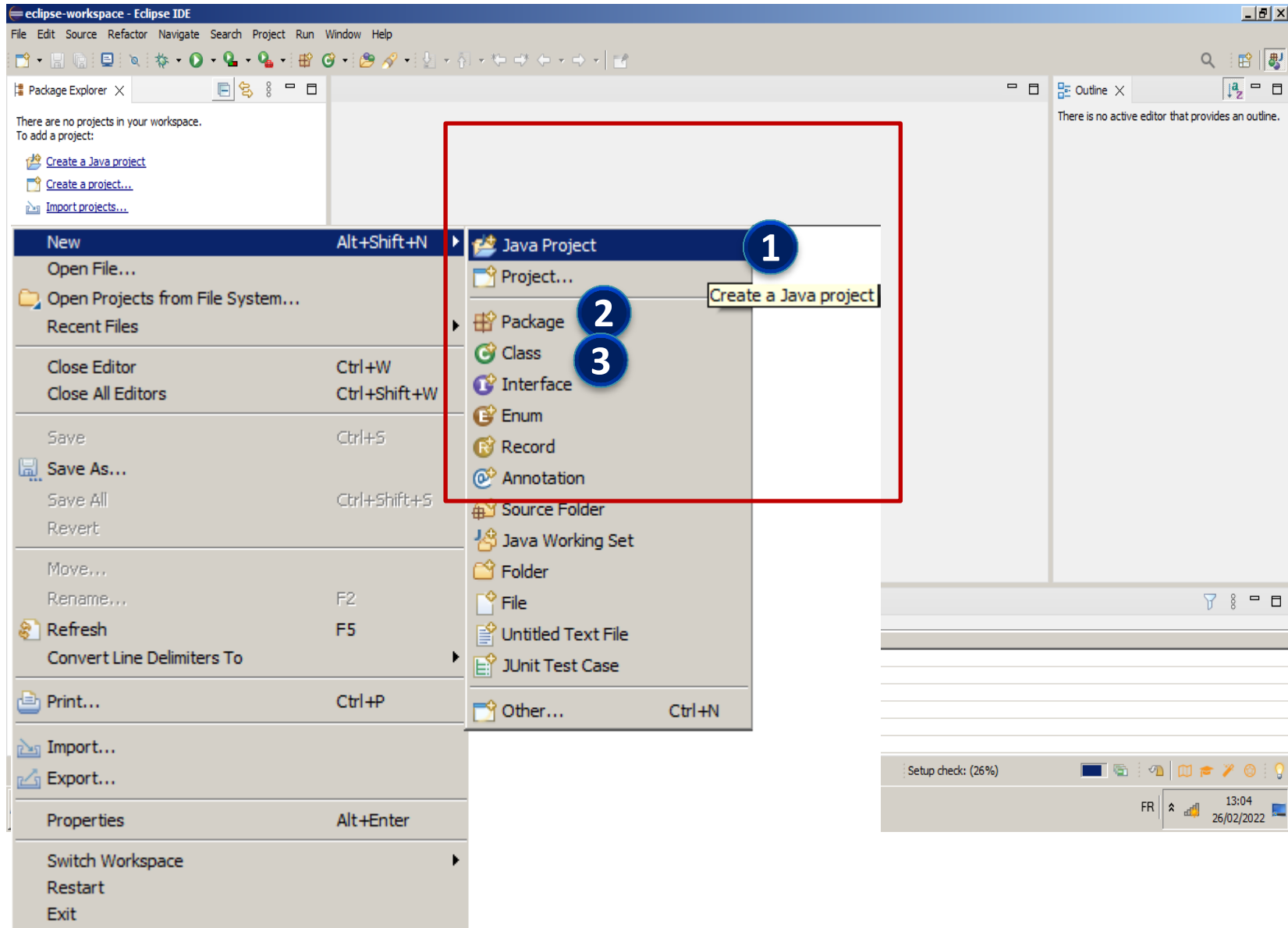
Commencer avec Eclipse



Commencer avec Eclipse (première programme)



Commencer avec Eclipse (première programme)



Commencer avec Eclipse Créer projet

New Java Project

Create a Java Project

Enter a project name.

Project name:

☒ Use default location

Location: [Browse...](#)

JRE

☒ Use an execution environment JRE:

☐ Use a project specific JRE:

☐ Use default JRE 'jdk-17.0.1' and workspace compiler preferences [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets [New...](#)

Working sets: [Select...](#)

Module

☒ Create module-info.java file

[?](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

Commencer avec Eclipse

New Java Class

Java Class
Create a new Java class.

Source folder: Browse...

Package: Browse...

☐ Enclosing type: Browse...

Name:

Modifiers:
☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?

☒ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Finish Cancel

Commencer avec Eclipse

```
1 package bonjour;  
2  
3 public class Bonjour {  
4  
5     public static void main(String[] args) {  
6  
7  
8     }  
9  
10 }
```

Nom du package en minuscule

Nom de la classe en majuscule

Nom de la méthode Main()



Entres et Sorties

input and output

Commencer avec Eclipse: Afficher un message

```
public class Bonjour {  
  
    public static void main(String[] args) {  
  
        System.out.println("Bonjour");  
  
    }  
  
}
```

Commencer avec Eclipse: Lire une valeur

```
package bonjour;
import java.util.Scanner;
public class Bonjour {

    public static void main(String[] args) {

        System.out.print("Donner votre nom SVP:");
        Scanner scanner=new Scanner(System.in);
        String name=scanner.nextLine();
        System.out.println("Bienvenu: "+name);

    }

}
```

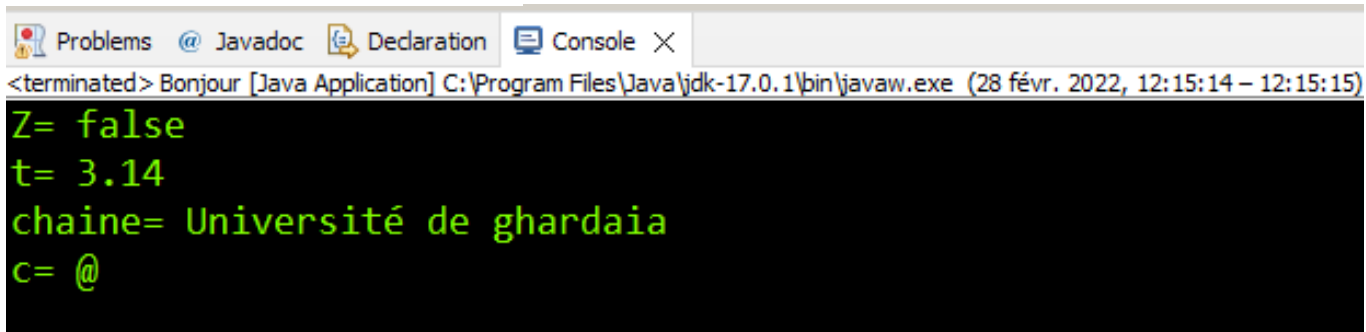


Variables **Boucles** **Conditions** **dans JAVA**

Les variables :

```
public class Bonjour {  
  
    public static void main(String[] args) {  
  
        // Variables :  
        boolean z=false;           //boolean 1 bit (true or false)  
        int x=123;                  //int      4 bytes=octets  
        double t=3.14;              //double  8 bytes=octets  
        char c='@';                 //char    2 bytes=octets  
        String chaine="Université de ghardaia"; //string  varies  
        // Affichage :  
        System.out.println("Z= "+z);  
        System.out.println("t= "+t);  
        System.out.println("chaine= "+chaine);  
        System.out.println("c= "+c);  
    }  
}
```

Résultat d'exécution:



The screenshot shows a Java IDE window with tabs for Problems, Javadoc, Declaration, and Console. The Console tab is active, displaying the output of the program. The output is as follows:

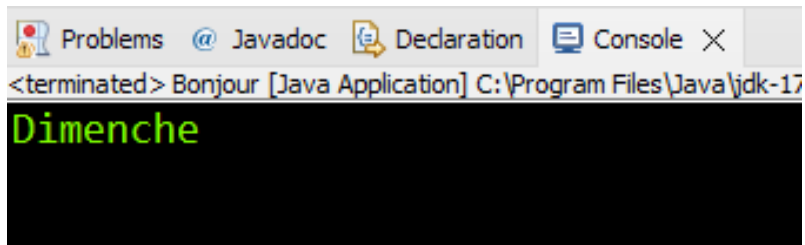
```
<terminated> Bonjour [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (28 févr. 2022, 12:15:14 – 12:15:15)  
Z= false  
t= 3.14  
chaine= Université de ghardaia  
c= @
```

Les opérateurs:

Opérateur	Description
+	Addition
-	Soustraction
*	Produit
/	Quotient de division euclidienne
%	Reste de division euclidienne
&&	"et" conditionnel
	"ou" conditionnel
=	Affectation
==	Comparaison de valeurs
< <=	Inférieur - inférieur ou égal
> >=	Supérieur - supérieur ou égal
!=	Différent de
!	"non" logique
~	Complément à deux (inverse le signe)
&	"et" arithmétique
	"ou" arithmétique
^	"ou" exclusif (xor) arithmétique
<<	Décalage à gauche
>>	Décalage à droite signé
>>>	Décalage à droite non signé

L'instruction: if else

```
public class Bonjour {  
  
    public static void main(String[] args) {  
        int jour=2;  
        if (jour==1) {  
            System.out.println("Samedi");  
        }  
        else if (jour==2)  
        {  
            System.out.println("Dimanche");  
        }  
    }  
}
```

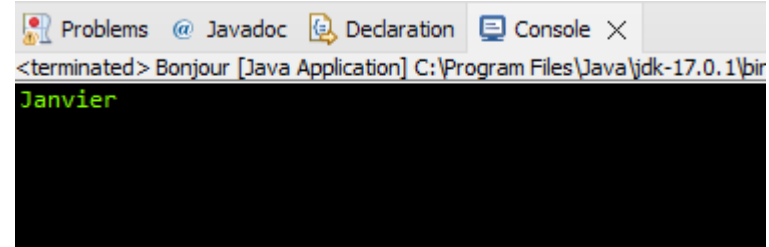


L'instruction: switch

```
package bonjour;

public class Bonjour {

    public static void main(String[] args) {
        int mois=1;
        switch(mois)
        {
            case 1:
                System.out.println("Janvier");
                break;
            case 2:
                System.out.println("Février");
                break;
            case 3:
                System.out.println("Mars");
                break;
            case 4:
                System.out.println("Avril");
                break;
            default:
                System.out.println("Erreur: nombre du mois erroné");
        }
    }
}
```

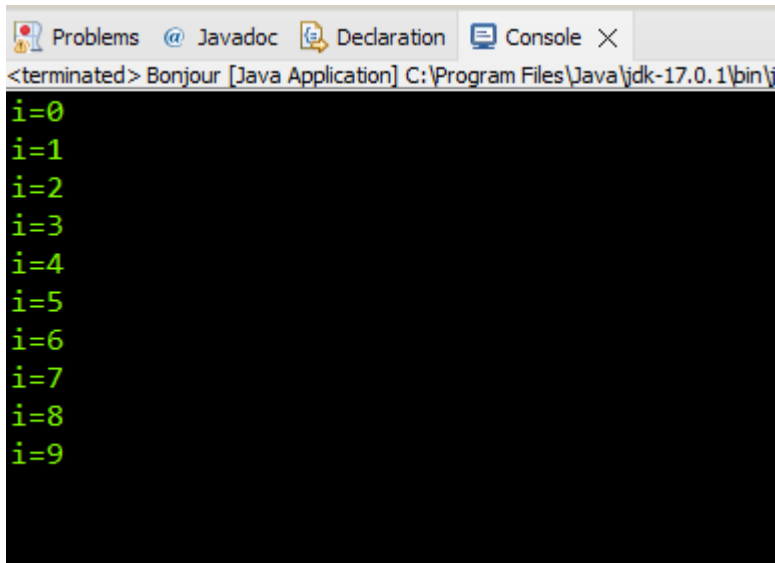


Boucle: for

```
package bonjour;

public class Bonjour {

    public static void main(String[] args) {
        for (int i=0; i<10; i++) {
            System.out.println("i="+i);
        }
    }
}
```



The screenshot shows a Java IDE window with the title bar "Problems Javadoc Declaration Console". The console output is as follows:

```
<terminated> Bonjour [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\j
i=0
i=1
i=2
i=3
i=4
i=5
i=6
i=7
i=8
i=9
```

Boucle: while

```
package bonjour;
```

```
public class Bonjour {
```

```
    public static void main(String[] args) {
```

```
        int i=0;
```

```
        while (i<10) {
```

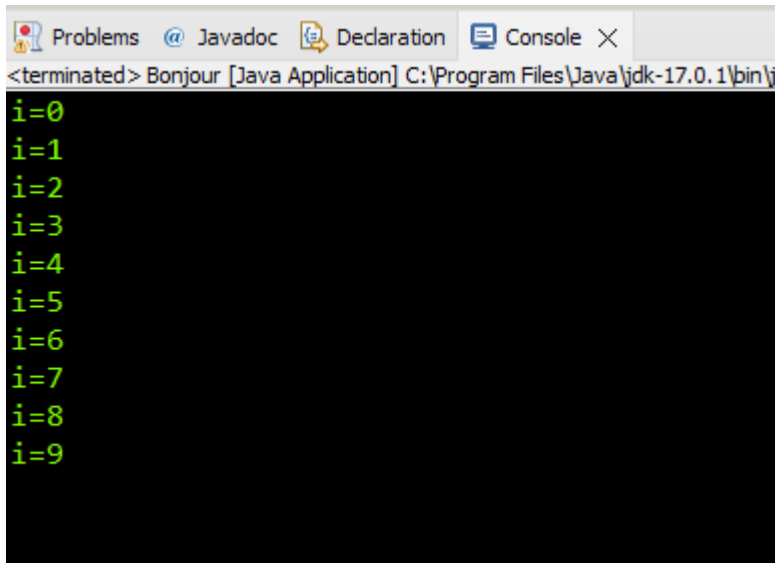
```
            System.out.println("i="+i);
```

```
            i++; // i=i+1;
```

```
        }
```

```
    }
```

```
}
```



The screenshot shows a Java IDE window with a console tab. The console output displays the values of 'i' from 0 to 9, each on a new line, confirming that the 'while' loop executed 10 times as intended. The window title bar includes 'Problems', 'Javadoc', 'Declaration', and 'Console' tabs. The console text is as follows:

```
<terminated> Bonjour [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\j  
i=0  
i=1  
i=2  
i=3  
i=4  
i=5  
i=6  
i=7  
i=8  
i=9
```

Boucle: do while

```
package bonjour;
```

```
public class Bonjour {
```

```
    public static void main(String[] args) {
```

```
        int i=0;
```

```
        do {
```

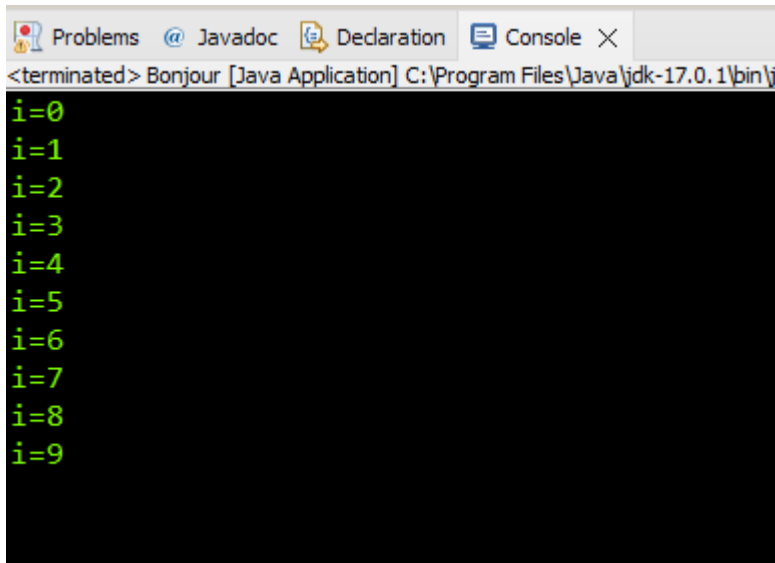
```
            System.out.println("i="+i);
```

```
            i++;
```

```
        } while (i<10);
```

```
    }
```

```
}
```

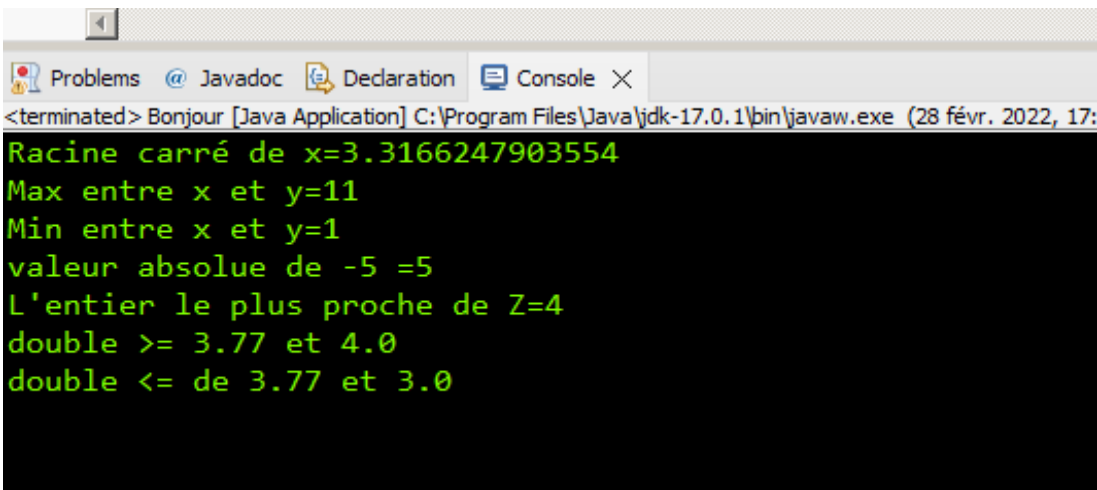


The screenshot shows a Java IDE window with a console tab. The console output displays the values of 'i' from 0 to 9, each on a new line, confirming that the 'do while' loop executed 10 times. The window title bar indicates the application is 'Bonjour [Java Application]' and the path is 'C:\Program Files\Java\jdk-17.0.1\bin\j'.

```
<terminated> Bonjour [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\j  
i=0  
i=1  
i=2  
i=3  
i=4  
i=5  
i=6  
i=7  
i=8  
i=9
```

Quelque fonctions mathématique:

```
package bonjour;  
import java.lang.Math;  
public class Bonjour {  
  
    public static void main(String[] args) {  
        int x=11;  
        int y=11 % 2; // modulo  
        double z=3.77;  
        int w=-5;  
        System.out.println("Racine carré de x="+Math.sqrt(x));  
        System.out.println("Max entre x et y="+Math.max(x, y));  
        System.out.println("Min entre x et y="+Math.min(x, y));  
        System.out.println("valeur absolue de "+w+" =" +Math.abs(w));  
        System.out.println("L'entier le plus proche de Z="+Math.round(z));  
        System.out.println("double >= "+z+" et "+Math.ceil(z));  
        System.out.println("double <= de "+z+" et "+Math.floor(z));  
    }  
}
```



The screenshot shows a Java IDE window with a console tab. The title bar indicates the application is 'Bonjour [Java Application]' and the path is 'C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe'. The console output displays the results of the program's execution, including the square root of 11, the maximum and minimum of 11 and 1, the absolute value of -5, the rounded value of 3.77, and the ceiling and floor of 3.77.

```
<terminated> Bonjour [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (28 févr. 2022, 17:  
Racine carré de x=3.3166247903554  
Max entre x et y=11  
Min entre x et y=1  
valeur absolue de -5 =5  
L'entier le plus proche de Z=4  
double >= 3.77 et 4.0  
double <= de 3.77 et 3.0
```

Exercice1

Ecrire un programme en java qui permet de tester l'âge d'une personne saisi à l'écran, il lui affiche mineur si l'âge est inférieur à 18 ans et majeur sinon.

Exercice2

Ecrire un programme Java qui demande à l'utilisateur de saisir un nombre entier **n** et de lui afficher si le nombre tapé est premier ou non ?

Exercice3

Ecrire un programme java qui demande à l'utilisateur de saisir son nom et de lui afficher son nom avec le message de bienvenue

Exercice4

Ecrire un programme java qui demande à l'utilisateur de saisir un nombre entier et de lui afficher que le nombre est pair ou impair selon la valeur introduite

Exercice5

Ecrire un programme Java qui calcul la somme des 100 premiers entiers

Exercice6

Ecrire un programme Java qui demande à l'utilisateur de saisir un nombre entier n et lui affiche la somme des n premiers nombres entiers

Exercice7

Ecrire un programme Java qui demande à l'utilisateur de saisir un nombre entier n et de lui afficher successivement tous les nombres pairs qui sont inférieur ou égale à n

Améliorer le programme de façon qu'il affiche en plus le nombre des entiers pairs inférieur ou égale à n

Exercice8

Ecrire un programme Java qui demande à l'utilisateur de saisir un nombre entier n inférieur ou égale à 9 et de lui afficher la table de multiplication de ce nombre



Classes et objets

La notion d'objet

Un **objet** est une abstraction d'un élément du monde réel.

Il possède un ensemble d'**attributs** caractérisant son état, et un ensemble d'opérations (les **méthodes**) qui permettent son comportement.



Etudiant
Nom: Richard Prénom: Simon Age: 24 Sexe: M
Etudier() Transférer()



Etudiant
Nom: Thomas Prénom: Louis Age: 21 Sexe: M
Etudier() Transférer()



Etudiant
Nom: Emma Prénom: Vincent Age: 23 Sexe: F
Etudier() Transférer()



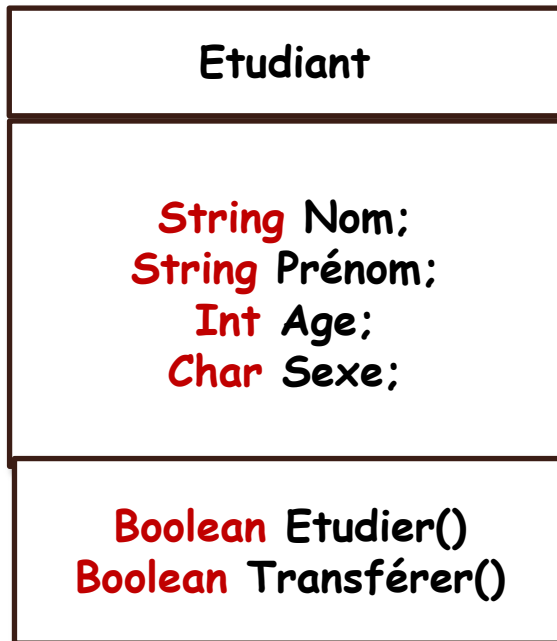
Etudiant
Nom: Alice Prénom: Klein Age: 22 Sexe: F
Etudier() Transférer()

La notion de classe

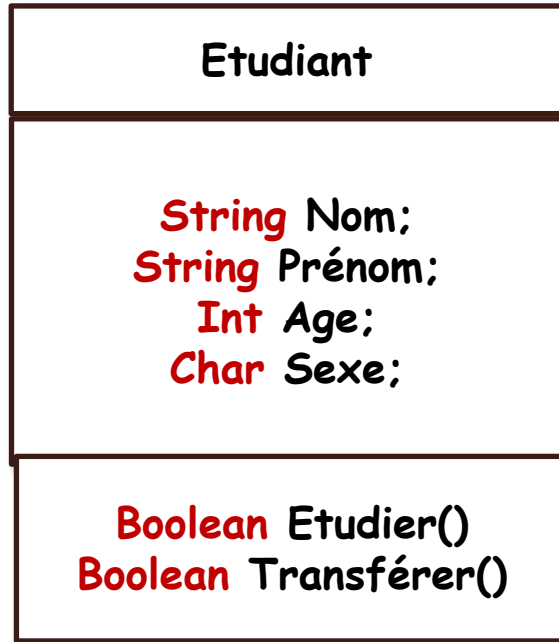
Une **classe**, est un type de données abstrait, caractérisé par des propriétés (ses attributs et ses méthodes) **communes à des objets**.

Elle permet de **créer ses objets** possédant ses propriétés

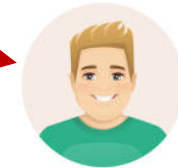
Un objet est l'**instance** d'une classe



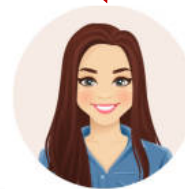
Exemple: Classe Etudiant



Etudiant
Nom: Richard Prénom: Simon Age: 24 Sexe: M
Etudier() Transférer()



Etudiant
Nom: Thomas Prénom: Louis Age: 21 Sexe: M
Etudier() Transférer()

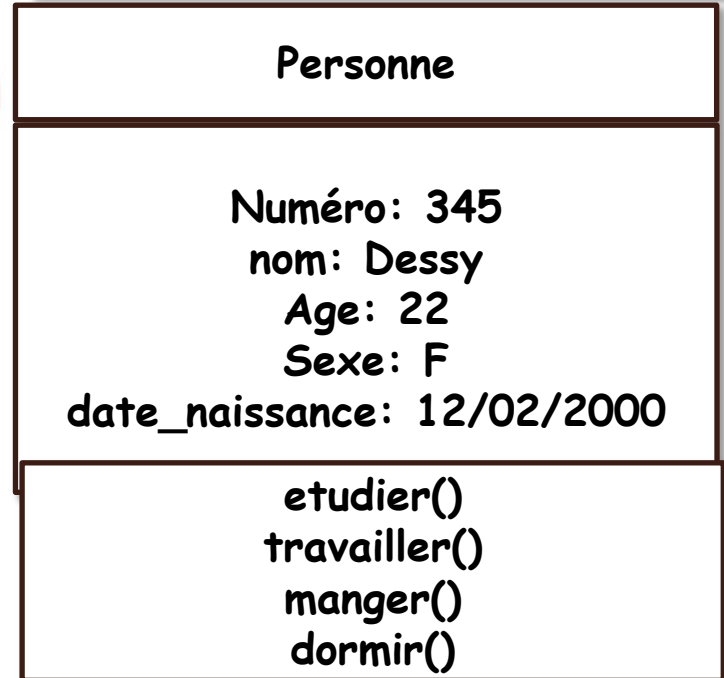
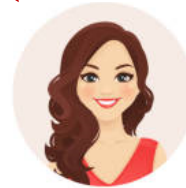
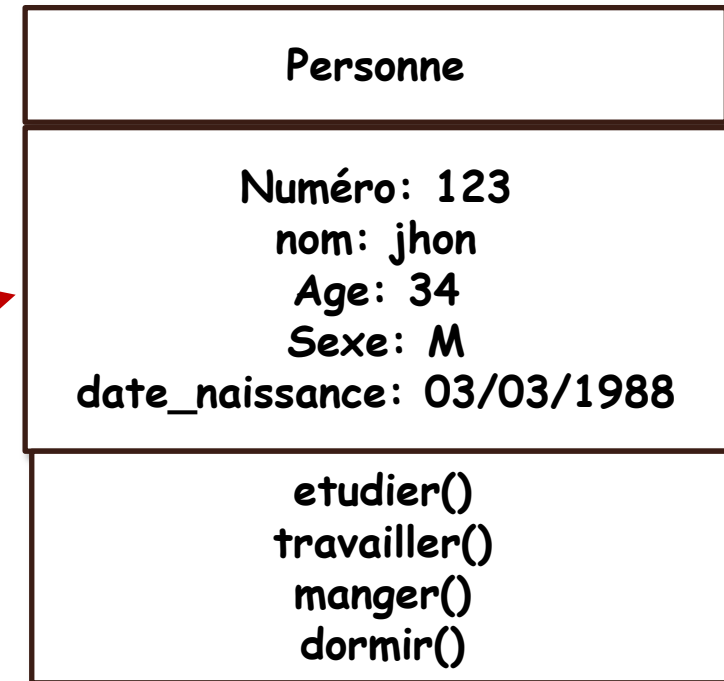
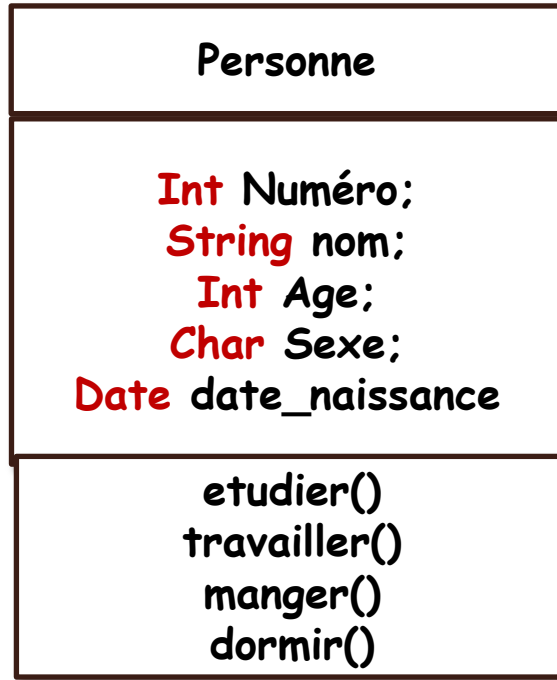


Etudiant
Nom: Alice Prénom: Klein Age: 22 Sexe: F
Etudier() Transférer()

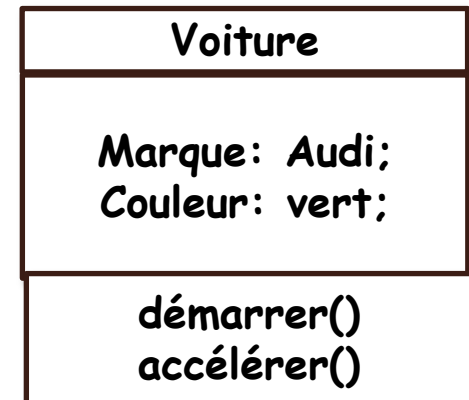
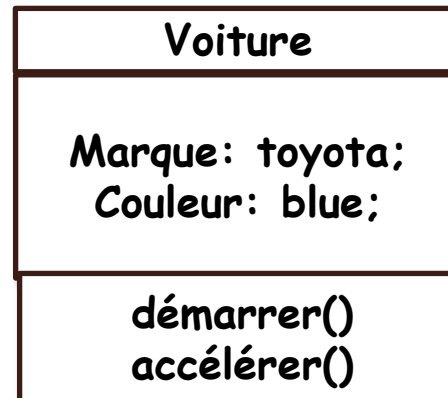
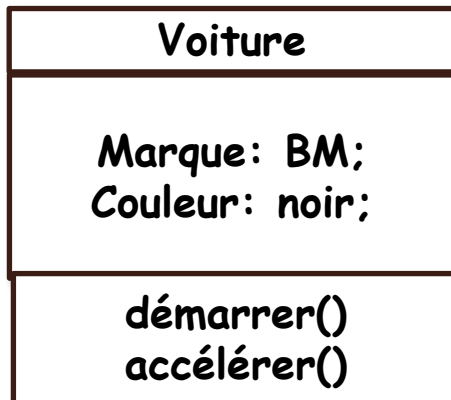
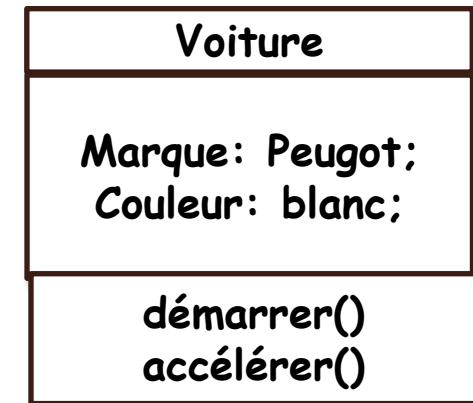
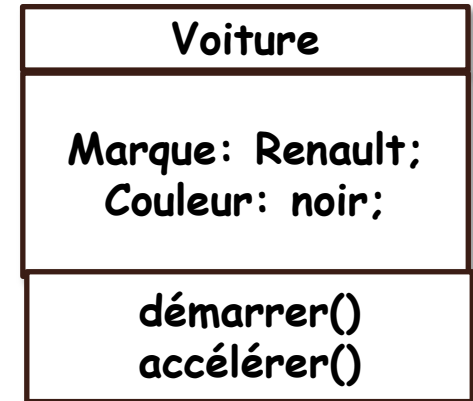
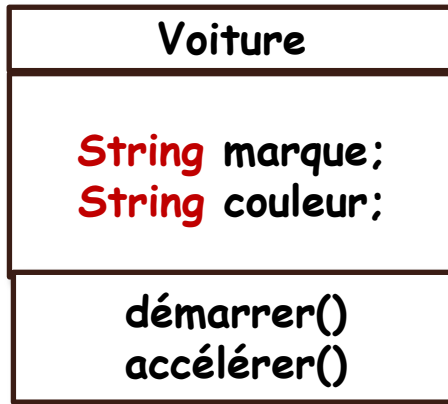


Etudiant
Nom: Emma Prénom: Vincent Age: 23 Sexe: F
Etudier() Transférer()

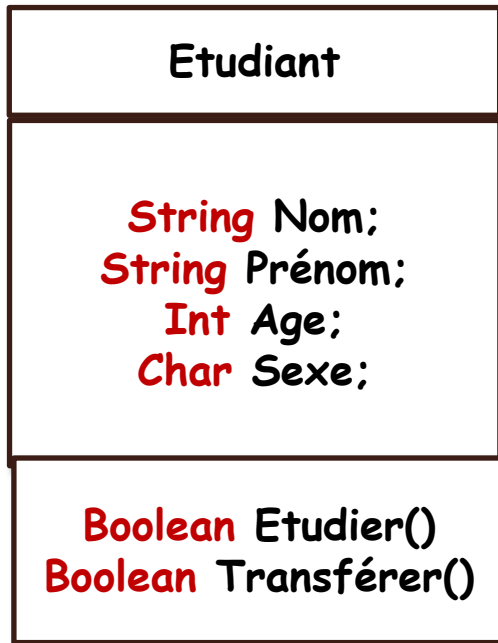
Exemple: Classe personne



Exemple: Classe Voiture



La notion de classe

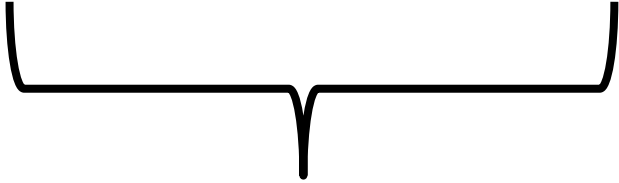


```
class nom_classe {  
    // Attributs  
    // Methodes  
    public static void main (String[] args) {  
  
    }  
}
```

```
package bonjour;  
  
public class Etudiant {  
    String name;  
    String prenom;  
    int age;  
    char sexe;  
    boolean etudier() {  
        return false;  
    }  
    boolean transférer() {  
        return false;  
    }  
    public static void main (String[] args) {  
  
    }  
}
```

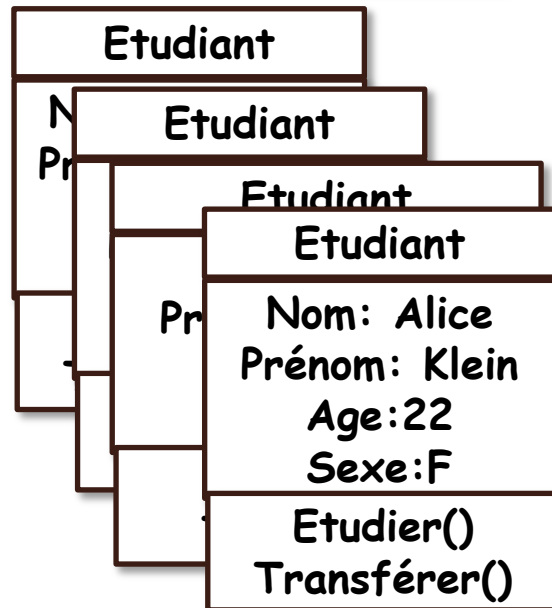
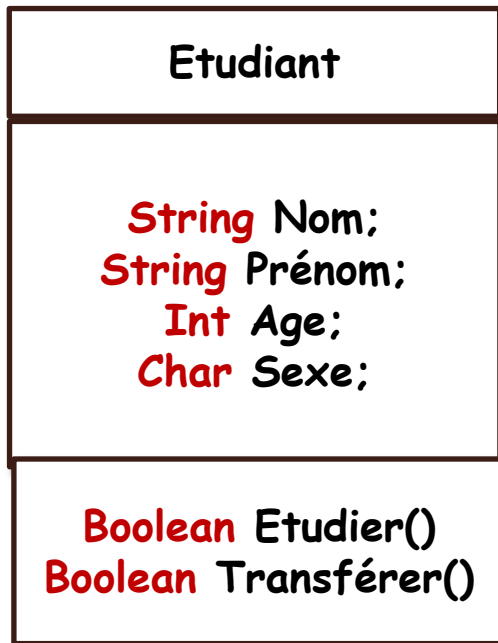
La notion de classe: Créer un objet

nom de la classe objet=new nom de la classe()



Constructeur

La notion de classe



```

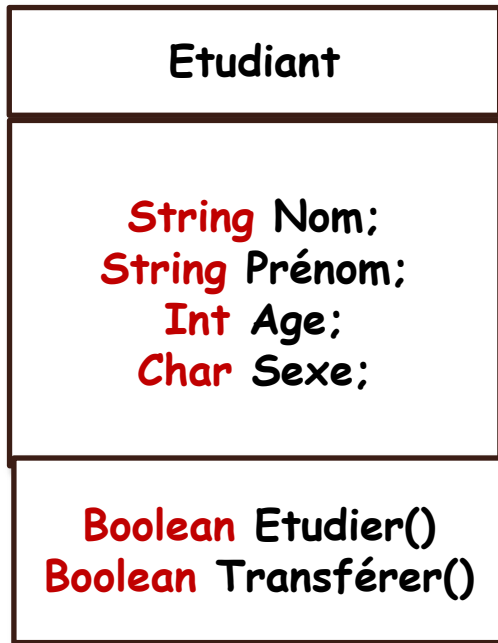
class nom_classe {
    // Attributs
    // Methodes

    public static void main (String[] args) {
        nom de la classe objet=new nom du
        constructeur()
    }
}

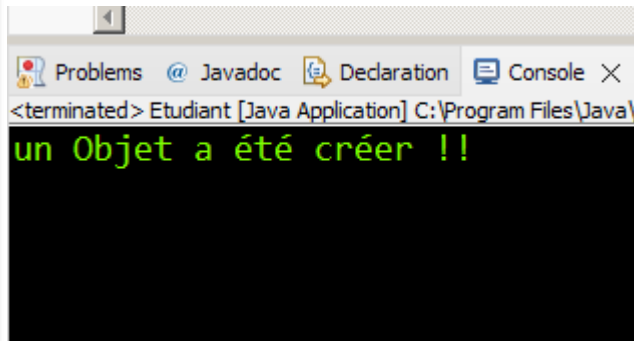
package bonjour;
public class Etudiant {
    String name;
    String prenom;
    int age;
    char sexe;
    boolean etudier() {
        return false;
    }
    boolean travailler() {
        return false;
    }
    public static void main (String[] args) {
        Etudiant e1=new Etudiant();
        Etudiant e2=new Etudiant();
        Etudiant e3=new Etudiant();
        Etudiant e4=new Etudiant();
    }
}

```

La notion de classe



Résultat d'exécution:

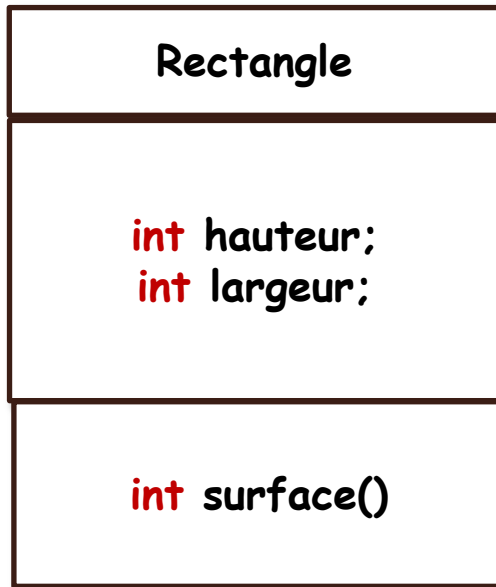


```
class nom_classe {
    // Attributs
    // Methodes

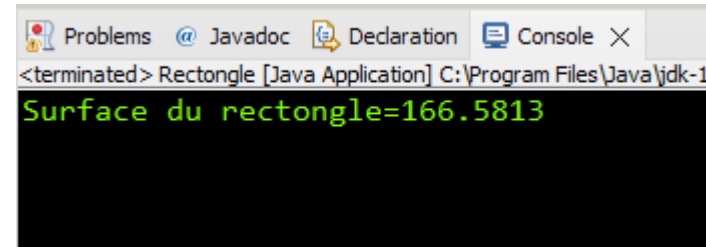
    public static void main (String[] args) {
        nom de la classe objet=new nom du
        constructeur()
    }
}
```

```
public class Etudiant {
    String name;
    String prenom;
    int age;
    char sexe;
    boolean etudier() {
        return false;
    }
    boolean transférer() {
        return false;
    }
    // Constructeur
    public Etudiant(){
        System.out.println("un Objet a été créer !!");
    }
    public static void main (String[] args) {
        Etudiant objet=new Etudiant();
    }
}
```

La notion de classe



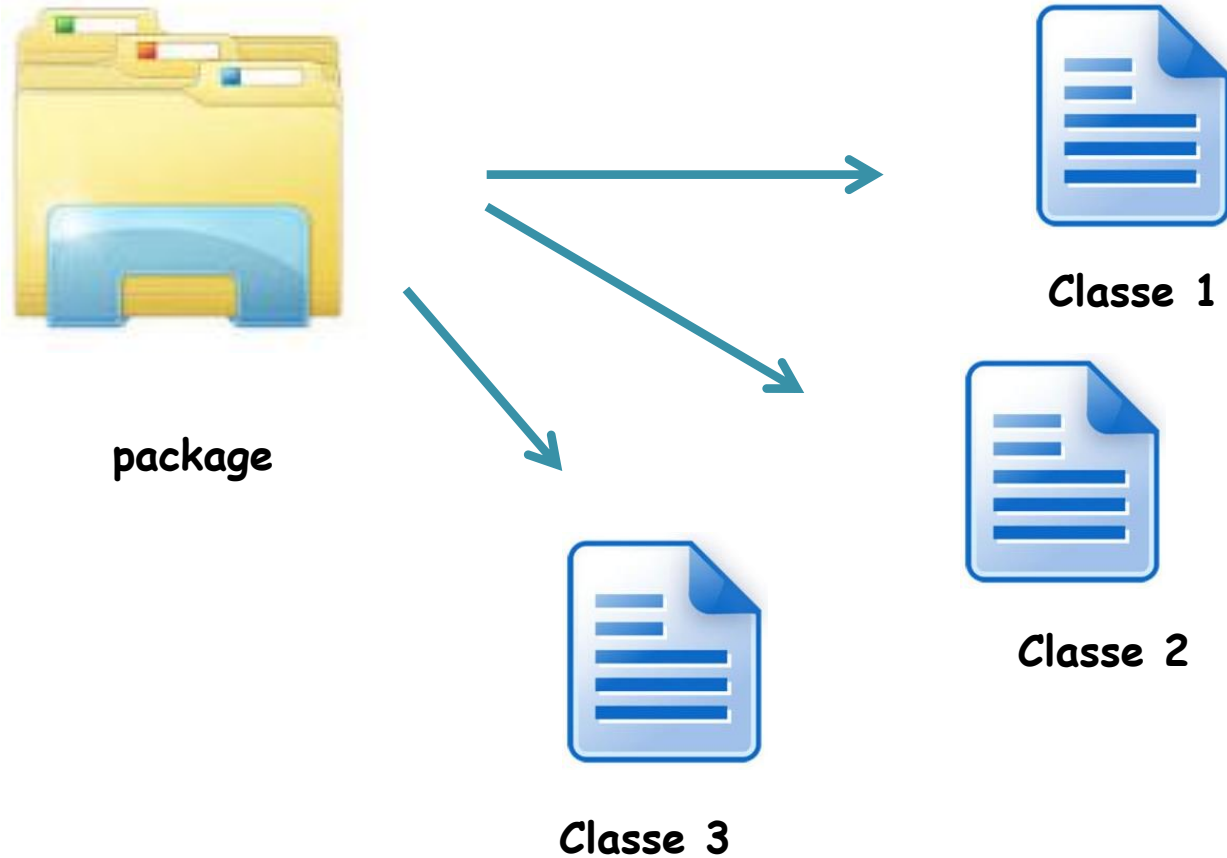
```
class nom_classe {
    // Attributs
    // Methodes
    public static void main (String[] args) {
        nom de la classe objet=new nom du
        constructeur()
    }
}
```



```
package bonjour;

public class Rectongle {
    double hauteur=11.01;
    double largeur=15.13;
    double surface () {
        return hauteur*largeur;
    }
    public static void main(String[] args) {
        Rectongle r=new Rectongle();
        System.out.println("Surface du rectangle="+ r.surface());
    }
}
```

La notion de package



Un package est une unité (un **fichier**) regroupant des **classes**

La notion de package

```
package bonjour;

public class Etudiant {
    String name;
    String prenom;
    int age;
    char sexe;

    boolean etudier() {
        return false;
    }
    boolean travailler() {
        return false;
    }
    public static void main (String[] args) {

    }
}
```

```
package bonjour;

public class Rectangle {

    int hauteur;

    int largeur;

    int surface () {

        return hauteur*largeur;
    }
    public static void main(String[] args) {

    }

}
```

Pour créer un package, il suffit de commencer le fichier source contenant les classes à regrouper par l'instruction **package** suivi du nom que l'on désire donner au package
Exemple: `package nom de package;`

La notion de package

Pour pouvoir utiliser une classe, par exemple, utiliser une de ces méthodes, il faut utiliser la clause "**import**", comme première instruction après le nom de package, comme l'exemple ci-dessus :

import nom de package.nom de la classe;

```
package bonjour;
import bonjour.Etudiant;
public class Rectangle {

    int hauteur;

    int largeur;

    int surface () {

        return hauteur*largeur;
    }
    public static void main(String[] args) {
        String name="mohammed";
        Etudiant objet=new Etudiant();
        objet.name=name;
        System.out.println("Etudier !: "+objet.etudier());
    }
}
```

La notion de package

le langage **java** est livré avec un grand ensemble de classes. L'ensemble des packages livrés par Java forme ce qu'on appelle l'API (Application Programming Interface). On les regroupe en packages nommés **java.*** (Vous pouvez consulter la documentation du JDK disponible pour connaître le contenu de ces classes).

package java.lang; (default package Variables et math fonctions)

package java.util; (Complément de java.lang)

package java.awt; (Pour les interfaces graphiques)

package javax.Swing; (Pour les interfaces graphiques)

Mis à part le package java.lang, tous les autres packages doivent être déclarés (mot clé import) pour pouvoir être utilisés



Constructeur

Destructeur

Constructeur

Un constructeur est une **méthode** spécifique. Cette méthode est appelée lors de la création d'un objet.

nom de la classe objet=**new** constructeur()

Quelques règles sur les constructeurs

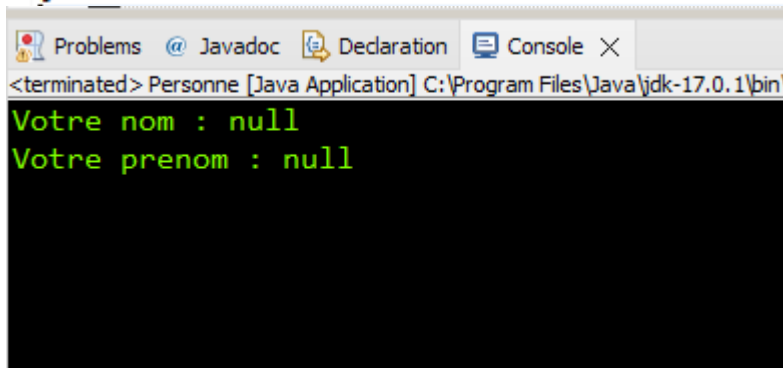
- Il prend obligatoirement le nom de la classe dans laquelle il est défini.
- Il n'a pas "de type de retour" et ne retourne" aucune valeur" et il n'est pas précédé par le mot clé "**void**".
- En cas d'absence de définition explicite, le compilateur crée un constructeur par défaut.

Constructeur

Il existe deux types de constructeur:

Par défaut (sans arguments):

```
public class Personne {  
    String nom;  
    String prenom;  
    void afficher() {  
        System.out.println("Votre nom : " + nom);  
        System.out.println("Votre prenom : " + prenom);  
    }  
    public static void main(String[] args) {  
        Personne p = new Personne();  
        p.afficher();  
    }  
}
```



The screenshot shows a Java IDE window with tabs for Problems, Javadoc, Declaration, and Console. The Console tab is active, displaying the output of the Java application. The text in the console is: <terminated> Personne [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\ and then two lines of output: Votre nom : null and Votre prenom : null.

```
<terminated> Personne [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\  
Votre nom : null  
Votre prenom : null
```

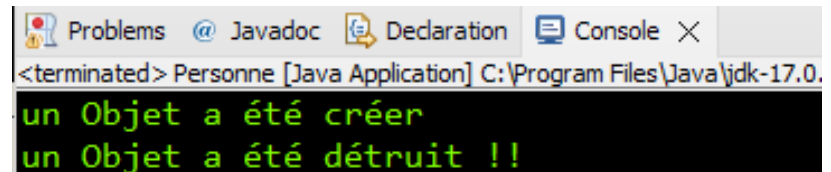
Destructeur

Le destructeur est une **méthode** particulière qui est appelée automatiquement à chaque destruction d'objet.

Pour définir un destructeur on doit utiliser la methode:

public void finalize()

```
public class Personne {  
    String nom;  
    String prenom;  
    // constructeur  
    public Personne() {  
        System.out.println("un Objet a été créer");  
    }  
    // Destructeur  
    public void finalize(){  
        System.out.println("un Objet a été détruit !!");  
    }  
    public static void main(String[] args) {  
        Personne p = new Personne();  
        p.finalize();  
    }  
}
```



Problems Javadoc Declaration Console X
<terminated> Personne [Java Application] C:\Program Files\Java\jdk-17.0.
un Objet a été créer
un Objet a été détruit !!

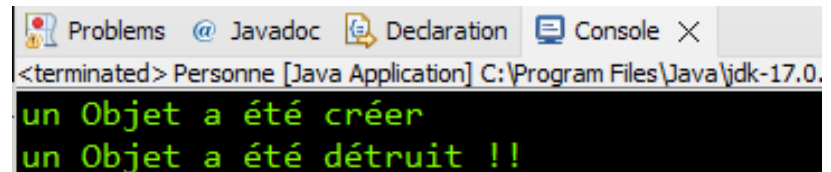
Destructeur

Le destructeur est une **méthode** particulière qui est appelée automatiquement à chaque destruction d'objet.

Pour définir un destructeur on doit utiliser la methode:

public void finalize()

```
public class Personne {
    String nom;
    String prenom;
    // constructeur
    public Personne() {
        System.out.println("un Objet a été créer");
    }
    // Destructeur
    public void finalize(){
        System.out.println("un Objet a été détruit !!");
    }
    public static void main(String[] args) {
        Personne p = new Personne();
        p.finalize();
    }
}
```



Problems Javadoc Declaration Console X
<terminated> Personne [Java Application] C:\Program Files\Java\jdk-17.0.
un Objet a été créer
un Objet a été détruit !!

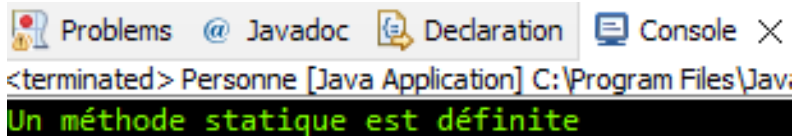


Méthodes et attributs statiques

Statique

Toute propriété (attribut ou méthode) est déclarer comme **static**, on peut l'utiliser sans créer un objet.

```
public class Personne {
    String nom;
    String prenom;
    // constructeur
    public Personne() {
        System.out.println("un Objet a été créer");
    }
    // Destructeur
    public void finalize(){
        System.out.println("un Objet a été détruit !!");
    }
    static void count() {
        System.out.println("Un méthode statique est définitive");
    }
    public static void main(String[] args) {
        Personne.count();
    }
}
```



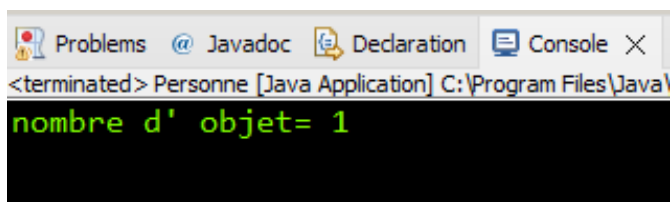
<terminated> Personne [Java Application] C:\Program Files\Java\

Un méthode statique est définitive

Statique

Toute propriété (attribut ou méthode) est déclarer comme **static**, on peut l'utiliser sans créer un objet.

```
public class Personne {  
    String nom;  
    String prenom;  
    static int nb_objet=1;  
    // constructeur  
    public Personne() {  
        System.out.println("un Objet a été créer");  
    }  
    // Destructeur  
    public void finalize(){  
        System.out.println("un Objet a été détruit !!");  
    }  
    static void count() {  
        System.out.println("nombre d' objet= "+nb_objet);  
    }  
    public static void main(String[] args) {  
        Personne.count();  
    }  
}
```



Statique

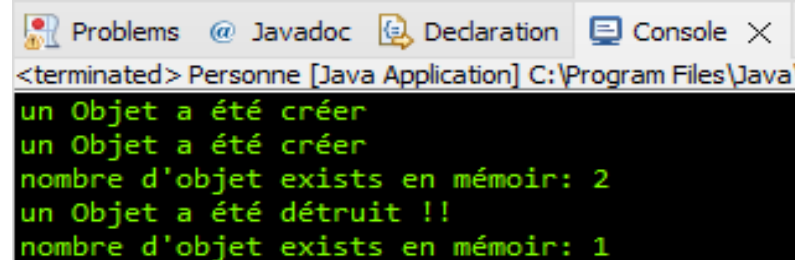
Toute propriété (attribut ou méthode) est déclarer comme **static**, on peut l'utiliser sans créer un objet.

- Les propriétés statiques d'une classe sont partagées par **toutes les instances** de cette classe.
- Si une méthode est statique, et si elle doit utiliser des attributs ou des méthodes de sa classe, il faut alors que ces **propriétés soient elles aussi déclarées static.**

Statique

Toute propriété (attribut ou méthode) est déclarer comme **static**, on peut l'utiliser sans créer un objet.

```
public class Personne {
    String nom;
    String prenom;
    static int nb_object=0;
    // constructeur
    public Personne() {
        System.out.println("un Objet a été créer");
        nb_object++;
    }
    // Destructeur
    public void finalize(){
        System.out.println("un Objet a été détruit !!");
        nb_object--;
    }
    static void count() {
        System.out.println("nombre d'objet exists en mémoire: "+nb_object);
    }
    public static void main(String[] args) {
        Personne p1 = new Personne();
        Personne p2 = new Personne();
        Personne.count();
        p1.finalize();
        Personne.count();
    }
}
```



Problems @ Javadoc Declaration Console X

<terminated> Personne [Java Application] C:\Program Files\Java\

```
un Objet a été créer
un Objet a été créer
nombre d'objet exists en mémoire: 2
un Objet a été détruit !!
nombre d'objet exists en mémoire: 1
```



Notion de visibilité
public
private
protected

Notion de visibilité

Selon le niveau de visibilité, les classes, les attributs et les méthodes peuvent être accessibles ou non depuis des classes du même paquetage ou depuis des classes d'autres paquetages.

Si on essaie de compiler une classe invoquant une classe, une méthode ou un attribut non accessible, le compilateur affiche un message d'erreur pour indiquer que l'entité en question n'est pas accessible.

Pour indiquer les degrés de visibilité, on utilise des modificateurs de visibilité. Les différents modificateurs sont indiqués par les mots clé :

- **private**
- **protected**
- **public**

Notion de visibilité

Ces modificateurs sont indiqués devant l'en-tête d'une classe ou d'une méthode ou devant le type d'un attribut.

Degré de visibilité nome de la classe/attribut/méthode

Exemple:

Public	class Etudiant
	class personne
Private	int matricule;
Protected	int age;

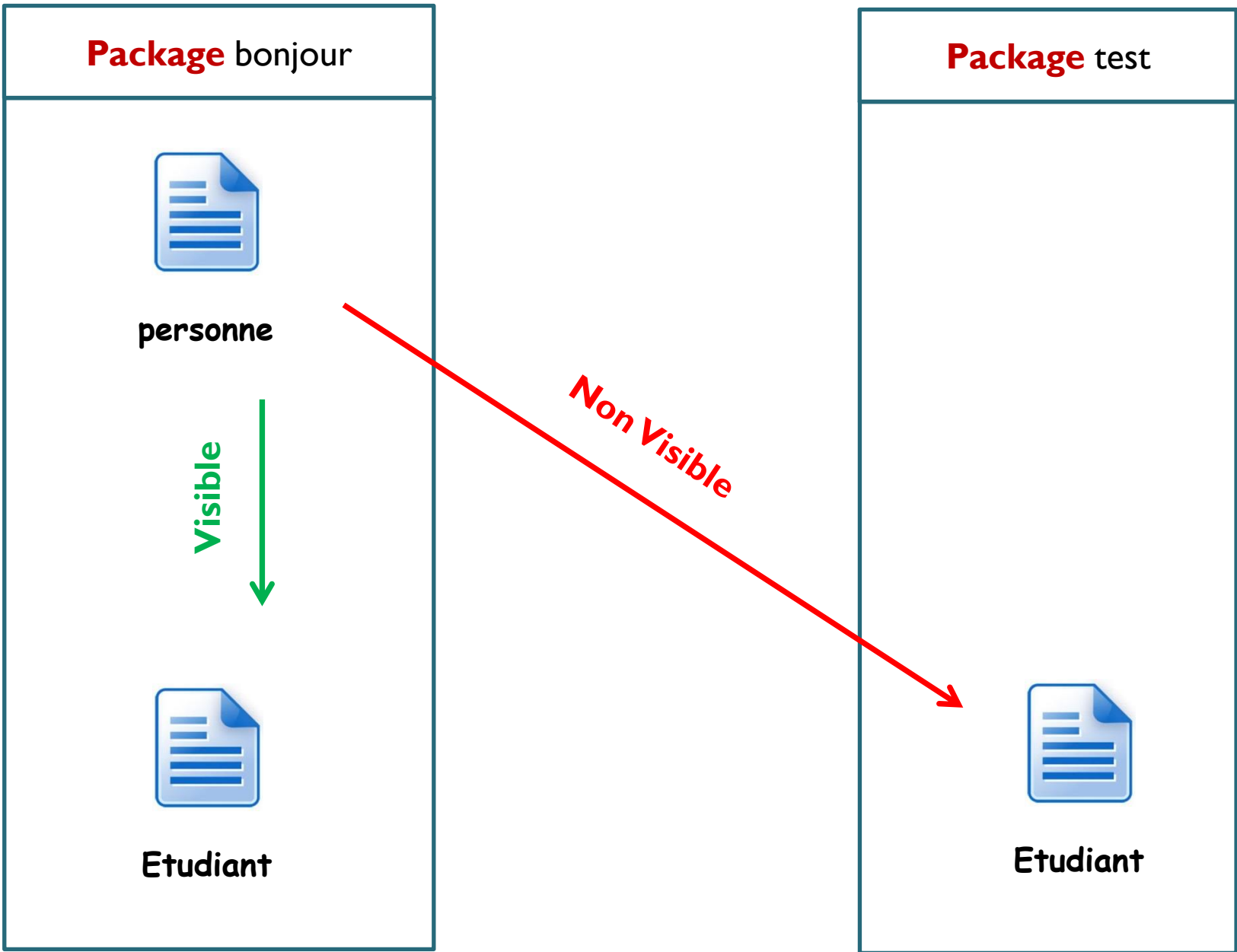
Lorsqu'il n'y a pas de modificateur, on dit que la visibilité est la visibilité **par défaut**

Visibilité de la classe

Une classe ne peut que :

- avoir la visibilité **par défaut** : elle n'est visible alors **que de son propre paquetage**.
- avoir la visibilité **public** : **elle est alors visible de partout**.

Class personne **par défaut**



Class personne par défaut

```
package bonjour;
```

```
class Personne {  
    String nom;  
    String Prenom;  
    int age;  
    public static void main(String[] args) {  
  
    }  
}
```

meme package

```
package bonjour;
```

```
import bonjour.Personne;
```

```
public class Etudiant {
```

```
    public static void main(String[] args) {  
        Personne p=new Personne();  
        p.nom="saidi";  
        p.Prenom="ahmed";  
        p.age=34;  
        System.out.println("Bienvenu "+p.nom+" "+p.Prenom);  
    }  
}
```

Problems @ Javadoc Declaration Console X
<terminated> Etudiant [Java Application] C:\Program Files\Java\
Bienvenu saidi ahmed

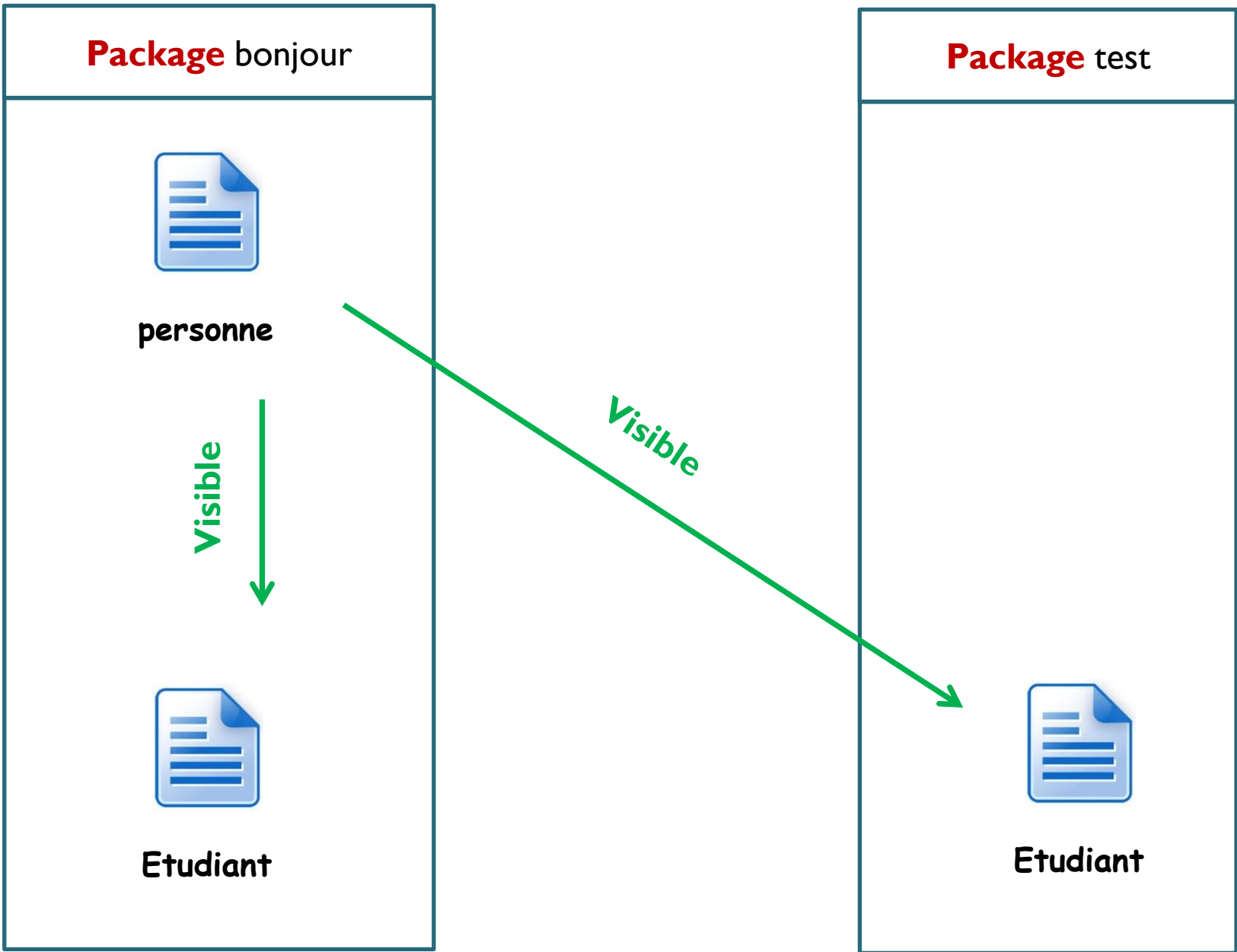
Class personne par défaut

```
package bonjour;  
  
class Personne {  
    String nom;  
    String Prenom;  
    int age;  
    public static void main(String[] args) {  
  
    }  
}
```

Pas le meme package

```
1 package test;  
2  
3 The type bonjour.Personne is not visible  
4  
5 public class Etudiant {  
6  
7     public static void main(String[] args) {  
8         Personne p=new Personne();  
9         p.nom="saidi";  
10        p.Prenom="ahmed";  
11        p.age=34;  
12        System.out.println("Bienvenu " +p.nom+" "+p.Prenom);  
13    }  
14 }
```


Class **personne** **public**



Class personne avec visibilité **public**

```
package bonjour;
```

```
public class Personne {  
    String nom;  
    String Prenom;  
    int age;  
    public static void main(String[] args) {  
  
    }  
}
```

Pas le meme
package

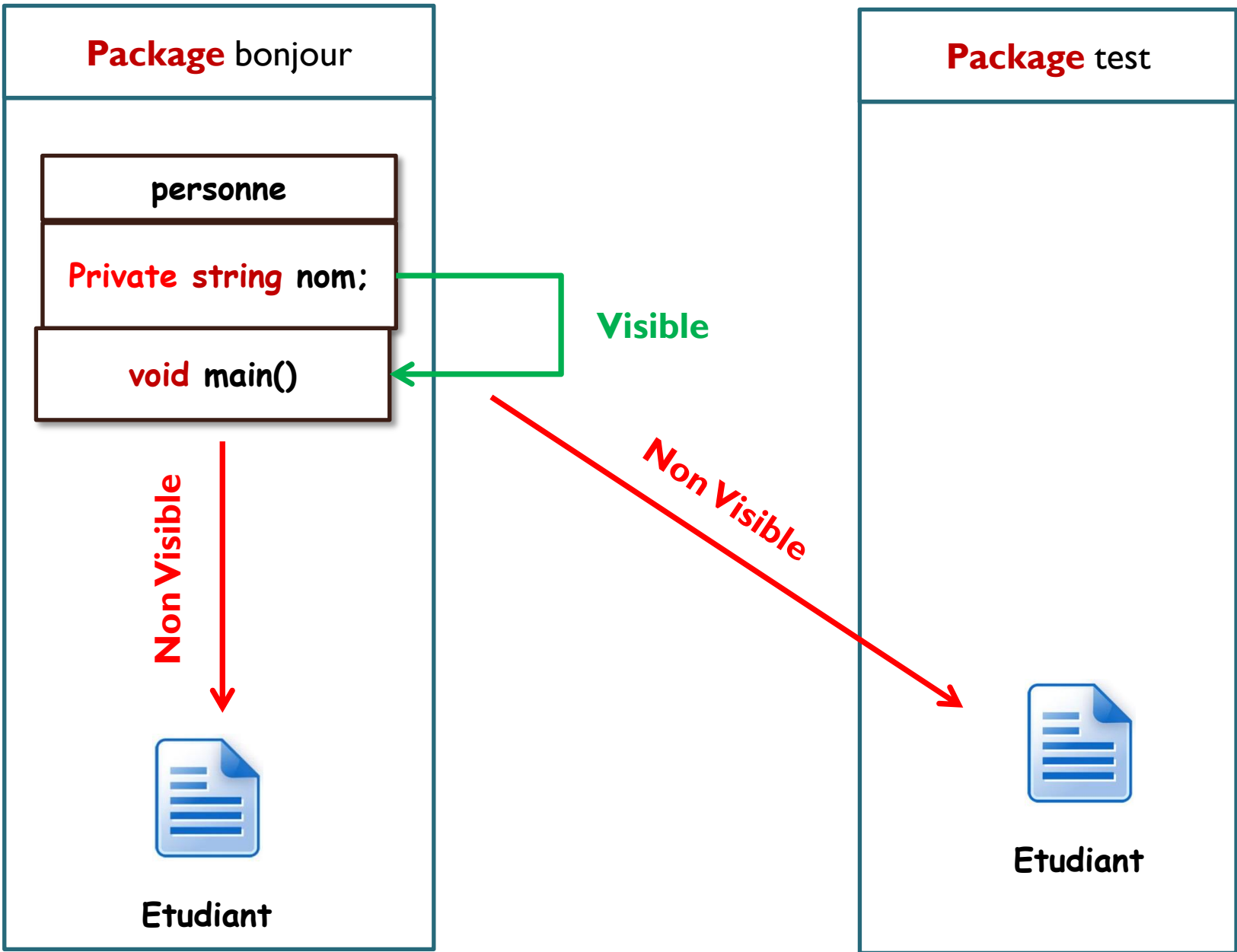
```
package test;
```

```
import bonjour.Personne;  
  
public class Etudiant {  
  
    public static void main(String[] args) {  
        Personne p=new Personne();  
  
    }  
}
```

visibilité des attributs et méthodes

- Si un **attributs/ méthodes** est **private** , il est accessible uniquement depuis sa propre classe ;

Attribue **private**



visibilité des attributs et méthodes

```
package bonjour;
```

```
class Personne {  
    private String nom;  
    protected String Prenom;  
    public int age;  
    int b;  
    public static void main(String[] args) {  
  
    }  
}
```

```
package bonjour;
```

```
import bonjour.Personne;
```

```
public class Etudiant {  
  
    public static void main(String[] args) {  
        Personne p=new Personne();  
        p.nom="saidi"; ✗  
        p.Prenom="ahmed"; ✓  
        p.age=34; ✓  
        p.b=1; ✓  
        System.out.println("Bienvenu "+p.nom+" "+p.Prenom);  
    }  
}
```

visibilité des attributs et méthodes

```
package bonjour;
```

```
class Personne {  
    private String nom;  
    protected String Prenom;  
    public int age;  
    int b;  
    public static void main(String[] args) {  
  
    }  
}
```

```
package test;
```

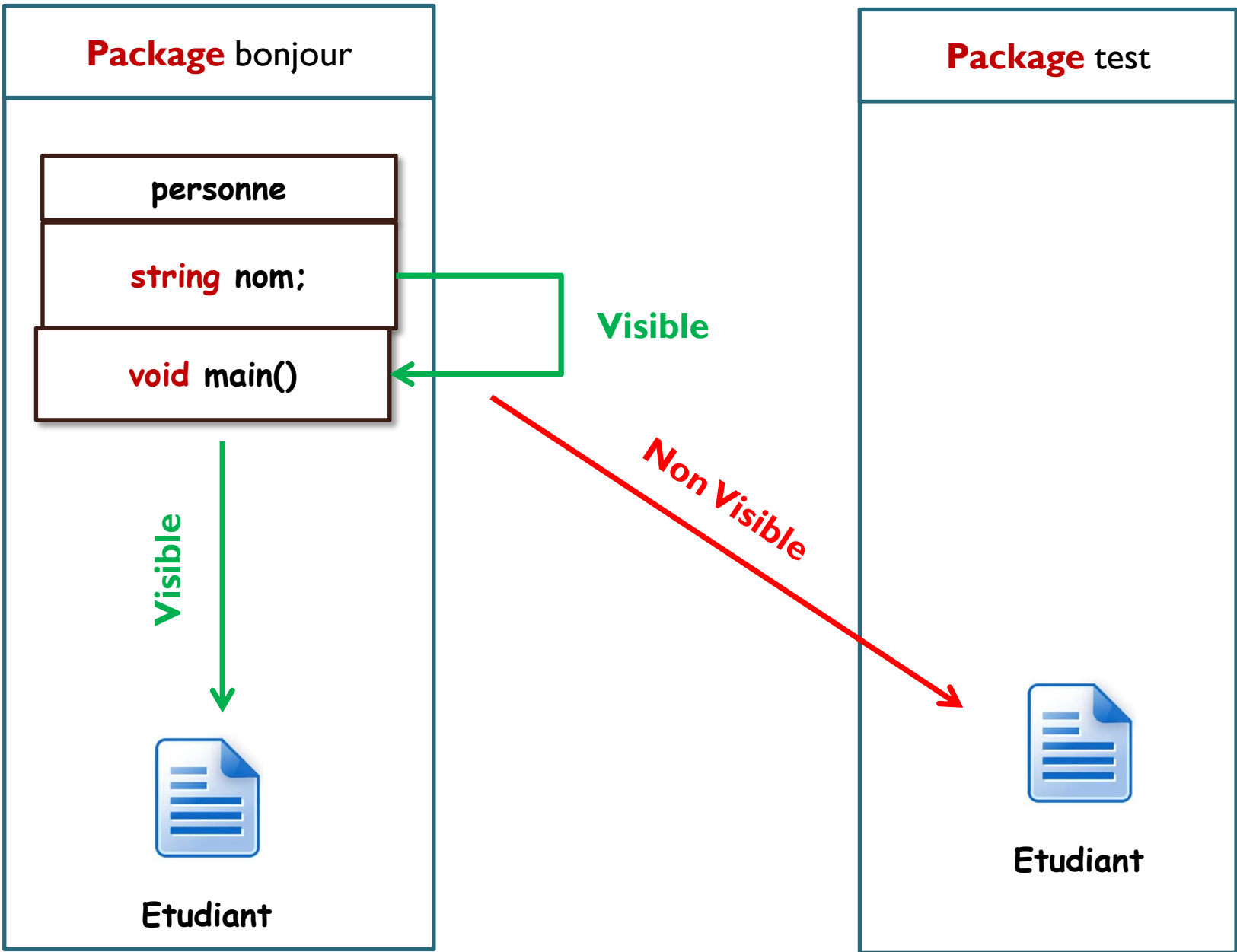
```
import bonjour.Personne; ✗
```

```
public class Etudiant {  
  
    public static void main(String[] args) {  
        Personne p=new Personne(); ✗  
        p.nom="saïdi"; ✗  
        p.Prenom="ahmed"; ✗  
        p.age=34; ✗  
        p.b=1; ✗  
  
    }  
}
```

visibilité des attributs et méthodes

- Si un **attributs/ méthodes** est **private** , il est accessible uniquement depuis sa propre classe ;
- Si un **attributs/ méthodes** est **par défaut** , il est accessible de partout dans le paquetage de sa classe;

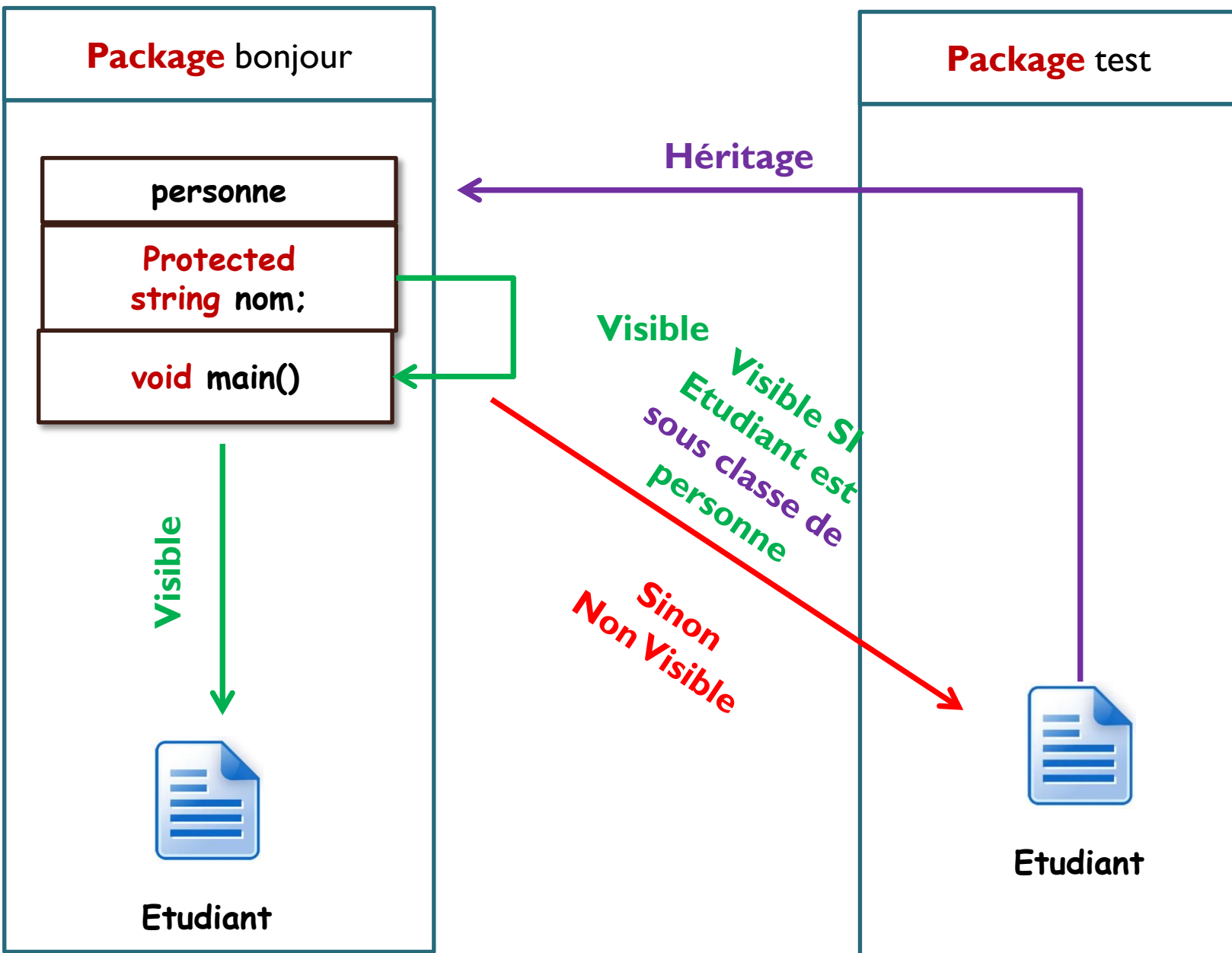
Attribue par défaut



visibilité des attributs et méthodes

- Si un **attribut/ méthode** est **private**, il est accessible uniquement depuis sa propre classe ;
- Si un **attribut/ méthode** est **par défaut**, il est accessible de partout dans le paquetage de sa classe ;
- Si un **attribut/ méthode** est **protected**, il est accessible de partout dans le paquetage de sa classe et dans les classes héritant de sa classe dans d'autres paquetages ;

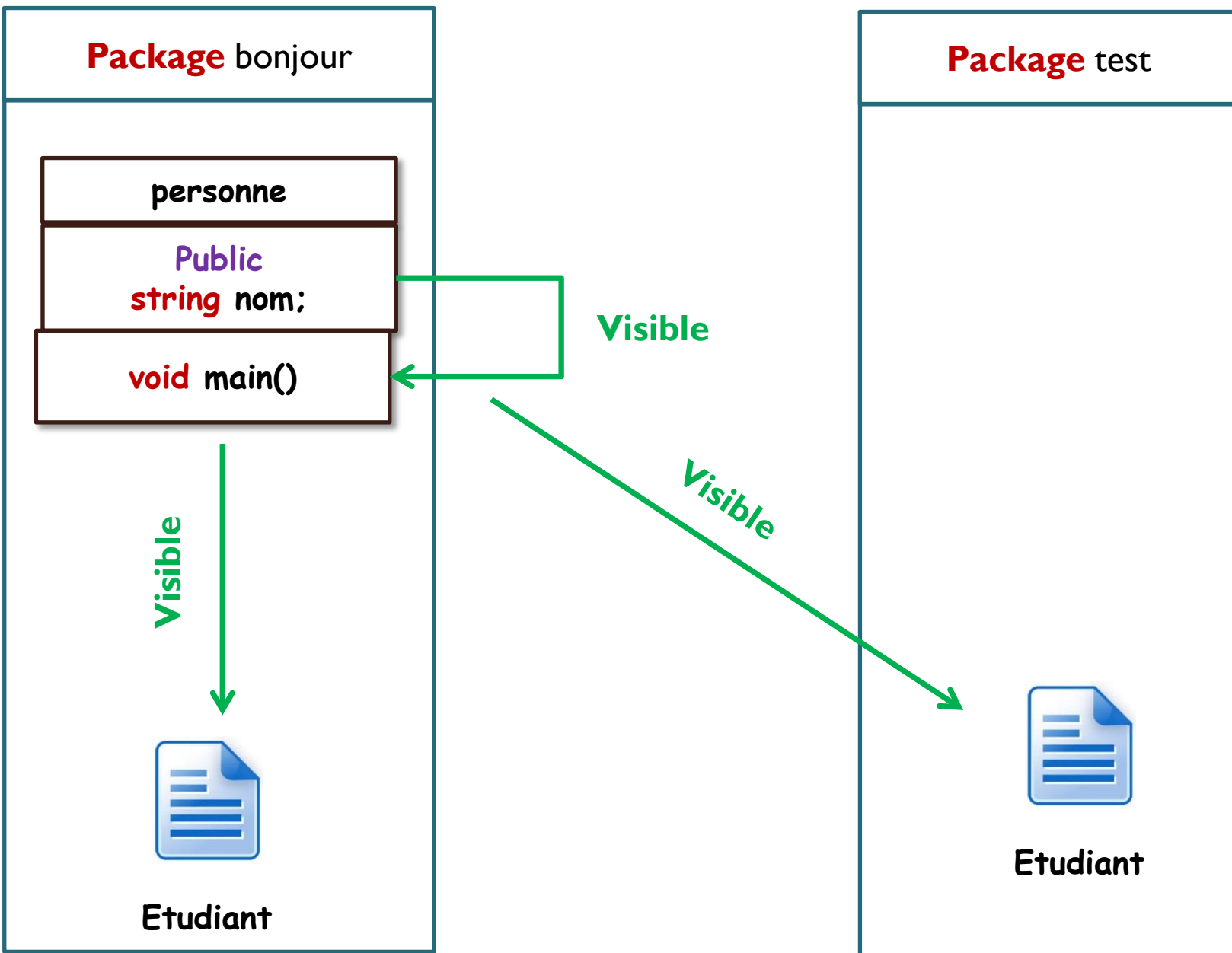
Attribue **protected**



visibilité des attributs et méthodes

- Si un **attribut/ méthode** est **private**, il est accessible uniquement depuis sa propre classe;
- Si un **attribut/ méthode** est **par défaut**, il est accessible de partout dans le paquetage de sa classe;
- Si un **attribut/ méthode** est **protected**, il est accessible de partout dans le paquetage de sa classe et dans les classes héritant de sa classe dans d'autres paquetages;
- Si un **attribut/ méthode** est **public**, il est accessible de partout dans le paquetage de sa classe et dans d'autres paquetages;

Attribue public



visibilité des attributs et méthodes

```
package bonjour;
```

```
public class Personne {  
    private String nom;  
    protected String Prenom;  
    public int age;  
    int b;  
    public static void main(String[] args) {  
  
    }  
}
```

```
package bonjour;
```

```
import bonjour.Personne;
```

```
public class Etudiant {
```

```
    public static void main(String[] args) {  
        Personne p=new Personne();  
        p.nom="saidi"; ✗  
        p.Prenom="ahmed"; ✓  
        p.age=34; ✓  
        p.b=1; ✓  
        System.out.println("Bienvenu "+p.nom+" "+p.Prenom);  
    }
```

```
}
```

visibilité des attributs et méthodes

```
package bonjour;
```

```
public class Personne {  
    private String nom;  
    protected String Prenom;  
    public int age;  
    int b;  
    public static void main(String[] args) {  
  
    }  
}
```

```
package test;
```

```
import bonjour.Personne;
```

```
public class Etudiant {  
  
    public static void main(String[] args) {  
        Personne p=new Personne();  
        p.nom="saidi"; ✗  
        p.Prenom="ahmed"; ✗  
        p.age=34; ✓  
        p.b=1; ✗  
  
    }  
}
```

Encapsulation

Dans la programmation orientée objet la définition **des niveaux de visibilité** est appelée « **Encapsulation** ».

L'encapsulation est un mécanisme consistant à **cacher** l'implémentation de l'objet, c'est-à-dire en empêchant l'accès aux données par un autre moyen que les services proposés.

Une classe doit rendre visible ce qui est **nécessaire** pour manipuler ses instances et rien d'autre.

L'encapsulation permet donc de garantir l'intégrité des données contenues dans l'objet.

Modificateur	private	default	protected	public
Accès depuis la classe	Oui	Oui	Oui	Oui
Accès depuis une classe du même package	Non	Oui	Oui	Oui
Accès depuis une sous-classe même package ou un autre	Non	Non	Oui	Oui
Accès depuis une classe du d'autre package	Non	Non	Non	Oui



Getters et Setters

Getters et Setters (Accesseurs et mutateurs)

Les accesseurs (getters) sont **des méthodes** qui va nous permettre d'afficher les variables de nos objets.

les mutateurs (setters) sont **des méthodes** qui va nous permettrons de modifier les variables de nos objets.

Getters et Setters (Accesseurs et mutateurs)

```
package bonjour;
```

```
public class Personne {  
    String nom;  
    String prenom;
```

```
}
```

```
package bonjour;
```

```
public class Etudiant {
```

```
    public static void main(String[] args) {  
        Personne p=new Personne();  
        p.nom="SAIDI";  
        p.prenom="AHMED";  
        System.out.println("NOM: "+p.nom);  
        System.out.println("PRENOM :"+p.prenom);
```

```
    }
```

```
}
```

Getters et Setters (Accesseurs et mutateurs)

```
package bonjour;

public class Personne {
    private String nom;
    private String prenom;

    public String get_nom() {
        return nom;
    }

    public String get_prenom() {
        return prenom;
    }

    public void set_nom(String _nom) {
        nom = _nom;
    }

    public void set_prenom(String _prenom) {
        prenom = _prenom;
    }
}
```

Getters et Setters (Accesseurs et mutateurs)

```
package bonjour;  
  
import bonjour.Personne;  
  
public class Etudiant {  
  
    public static void main(String[] args) {  
        Personne p=new Personne();  
        p.set_nom("Saïdi");  
        p.set_prenom("Ahmed");  
        System.out.println("Nom: "+p.get_nom());  
        System.out.println("Prenom: "+p.get_prenom());  
    }  
}
```

Getters et Setters (Accesseurs et mutateurs)

Les accesseurs (getters) sont **des méthodes** qui va nous permettre d'afficher les variables de nos objet.

les mutateurs (setters) sont **des méthodes** qui va nous permettrons de modifier les variables de nos objet.

Ces méthodes (accesseurs et mutateurs) doivent bien évidemment être **publiques** afin d'être accessibles depuis les autres classes (sans quoi il n'y aurait pas d'intérêt).

Un mutateur est toujours de type **void**, car il ne renvoie rien.

Un accesseur renvoie une variable et sera donc du type de celle-ci.

Mot clé « This »

- La clause « This » est généralement utilisée avec les getters et setters.
- Elle permet d'accéder aux attributs de la classe courante.
- Elle signifie « This class »

```
public class Personne {  
    private String nom;  
    private String prenom;  
  
    public String get_nom() {  
        return this.nom;  
    }  
  
    public String get_prenom() {  
        return this.prenom;  
    }  
    public void set_nom(String nom) {  
        this.nom = nom;  
    }  
  
    public void set_prenom(String prenom) {  
        this.prenom = prenom;  
    }  
}
```

La surcharge (**overloading**)

La surcharge (**overloading**) survient lorsque deux méthodes ou plus dans une classe ont le même nom de méthode mais des paramètres différents

```
public class Test {  
    public int somme(int x, int y) {  
        return x+y;  
    }  
    public int somme(int x, int y,int z) {  
        return x+y+z;  
    }  
    public double somme(double x, double y) {  
        return x+y;  
    }  
    public static void main(String[] args) {  
        Test t=new Test();  
        System.out.println("Somme deux entier: "+t.somme(4, 2));  
        System.out.println("Somme trois entiers: "+t.somme(4, 2,2));  
        System.out.println("Somme deux double: "+t.somme(1.5,2.5));  
    }  
}
```


La surcharge (**overloading**)

```
public class Point {  
    private int x;  
    private int y;  
  
    Public Point (int absc, int ord) { //constructeur  
        x = absc;  y = ord; }  
    public void deplacer (int dx, int dy) { //deplacer(int,int)  
        x = x + dx;  
        y = y + dy; }  
    public void deplacer (int dx) { //deplacer(int)  
        x = x + dx; }  
    public void deplacer (short dx) { //deplacer(short)  
        x = x + dx;  
    }  
  
    .....  
}
```

La surcharge (**overloading**)

```
public class Prog {  
  
    public static void main(String[] args) {  
        Point P = new Point (5,3) ; // appelle constructeur  
        P.afficher() ;  
        P.deplacer(-2,1);          // appelle deplacer(int,int)  
        P.afficher() ;  
        P.deplacer(3); // appelle deplacer(int)  
        P.afficher() ;  
        short s=1;  
        P.deplacer(s); // appelle deplacer(short)  
        P.afficher() ;  
        byte b=2;  
        P.deplacer(b); // appelle deplacer(short) après la conversion de b en short  
        P.afficher() ;  
    }  
}
```



Relation entre les classes

Héritage

personne

String nom;
String prenom
int age
Char Sexe;

void communiquer()
void déplacer()

Enseignant

String nom;
String prenom
int age
Char Sexe;
double Salaire

void communiquer()
void déplacer()
void Démissionner()
void Enseigner()
void Mutation()
void Consulter_comp()

Salariée

String nom;
String prenom
int age
Char Sexe;
double Salaire

void communiquer()
void déplacer()
void Démissionner()
void Mutation()
void Consulter_comp()

Etudiant

int Matricule;
String Nom;
String Prénom;
int Age;
Char Sexe;

void communiquer()
void déplacer()
Boolean Etudier()
Boolean Transférer()

Chef_département

String nom;
String prenom
int age
Char Sexe;
double Salaire

void communiquer()
void déplacer()
void Démissionner()
void Demander_Réunion()
void Mutation()
void Consulter_comp()

La relation d'héritage:

- L'héritage constitue l'un des fondements de base de la POO.
- Il permet le partage et la réutilisation de propriétés entre les objets.
- La relation d'héritage est une relation de **généralisation /spécialisation**.
- L'héritage permet de définir des classes sous forme d'une hiérarchie (**les fils héritent de leurs parents**).
- Une classe dérivée permet de préciser le comportement d'un sous-ensemble d'objet.
- Une **classe** peut avoir plusieurs **sous classes**.
- Une **classe** ne peut avoir qu'une seule **classe mère**.
- Supprime **les redondances** dans le code.

La relation d'héritage:

La classe dérivée:

- hérite de tous les membres de sa superclasse (**classe mère**).
- Elle permet de les spécialiser (**redéfinir**).
- Elle permet d'ajouter de nouveaux **attributs et/ou de nouvelles méthodes**.

Dans **JAVA**:

- il s'agit d'indiquer, dans la sous-classe, le nom de la superclasse dont elle hérite précéder par le mot réservé

extends

- Par défaut toutes classes Java hérite de la classe **Object**.

La relation d'héritage:

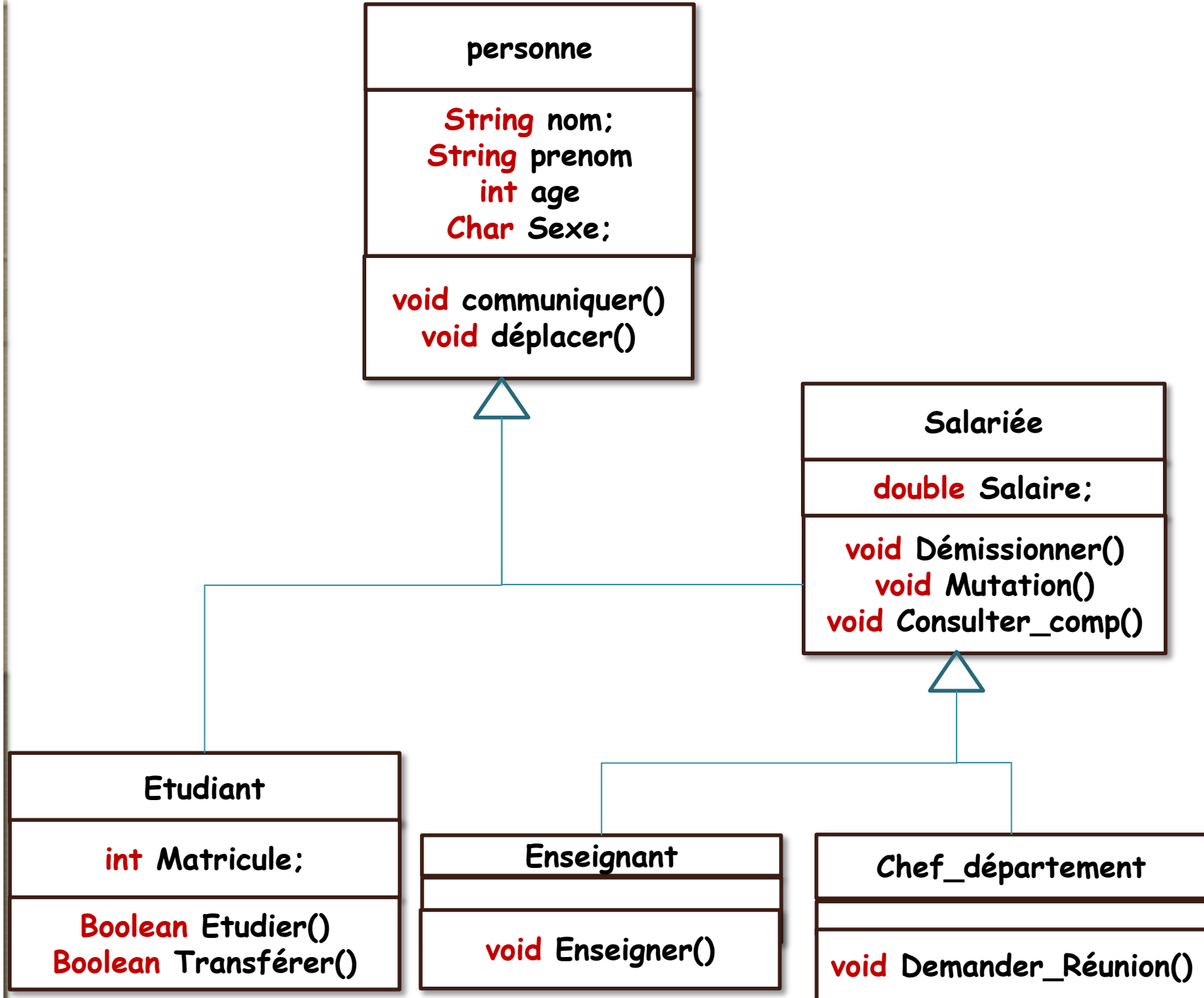
Classe_fille **extends** **Class_mère()**

Exemple:

Class Etudiant **extends** Personne

Class Salariée **extends** Personne

Class Enseignant **extends** Salariée

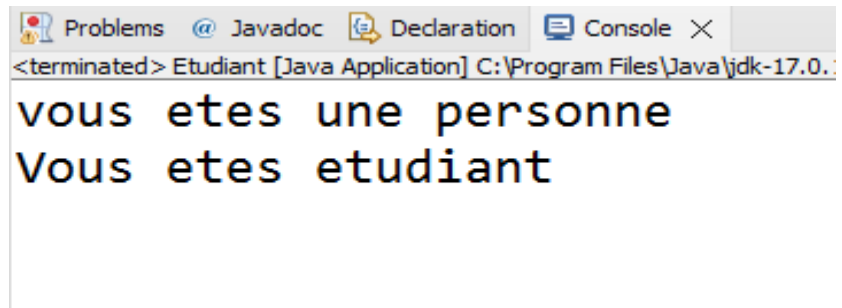



```
public class Personne {  
    String nom;  
    String prenom;  
    int age;  
    Personne(){  
        System.out.println("vous etes une personne");  
    }  
    public static void main(String[] args) {  
        Personne e=new Personne();  
    }  
}
```

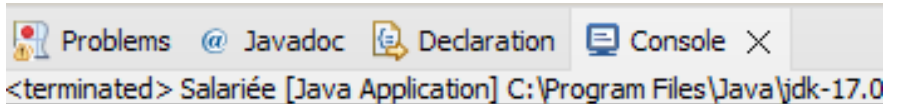
Problems Javadoc Declaration Console X
<terminated> Personne [Java Application] C:\Program Files\Java\jdk-1

vous etes une personne

```
public class Etudiant extends Personne{
    int matricule;
    Etudiant(){
        System.out.println("Vous etes etudiant");
    }
    public static void main(String[] args) {
        Etudiant e=new Etudiant();
    }
}
```



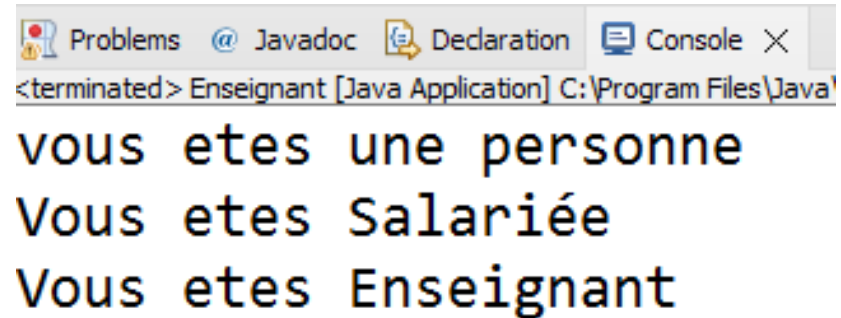
```
public class Salariée extends Personne{  
    double salaire;  
    Salariée(){  
        System.out.println("Vous etes Salariée");  
    }  
    public static void main(String[] args) {  
        Salariée e=new Salariée();  
    }  
}
```



The screenshot shows a console window with tabs for Problems, Javadoc, Declaration, and Console. The Console tab is active, displaying the output of the Java application: "<terminated> Salariée [Java Application] C:\Program Files\Java\jdk-17.0".

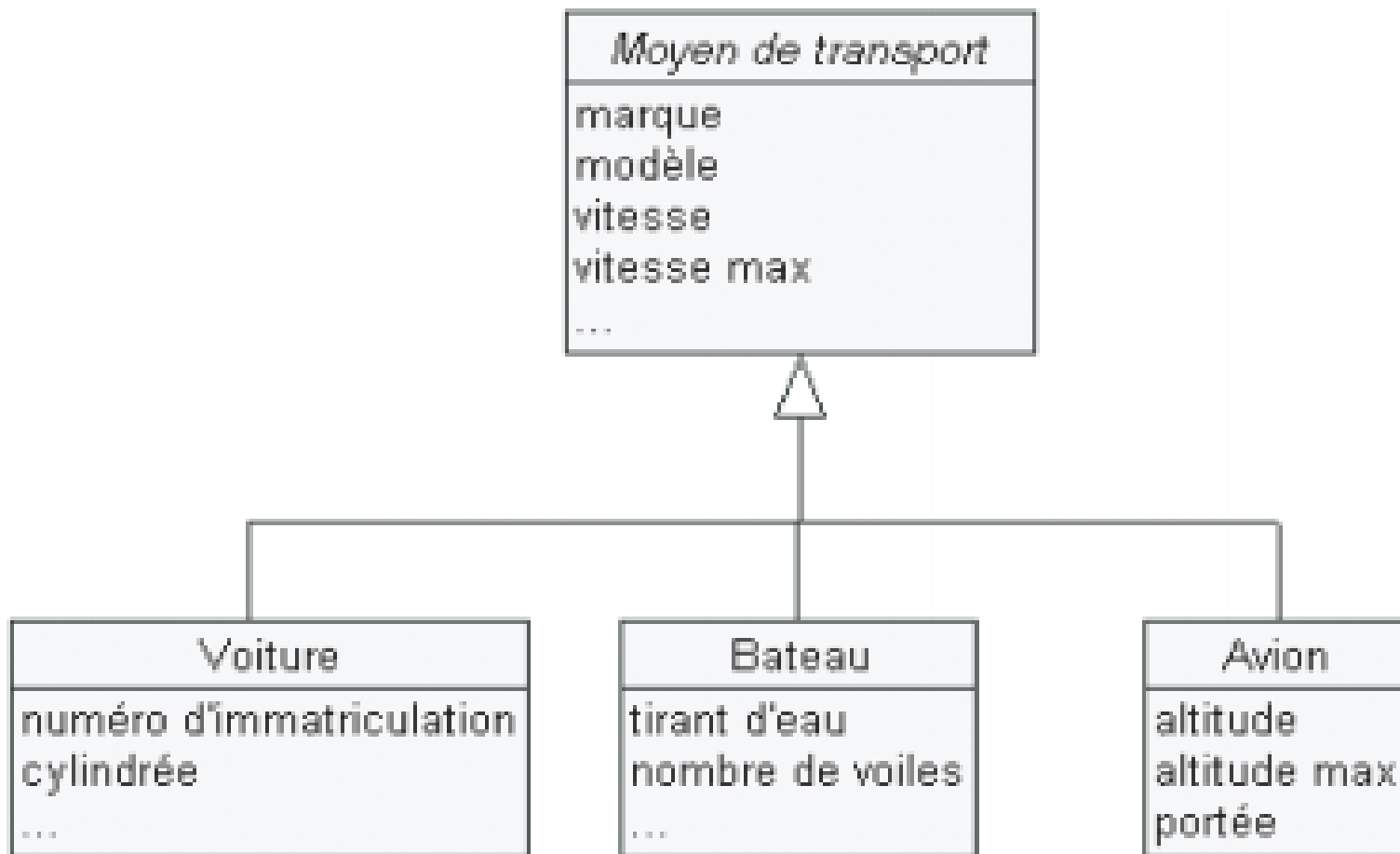
vous etes une personne
Vous etes Salariée

```
public class Enseignant extends Salariée{
    Enseignant(){
        System.out.println("Vous etes Enseignant");
    }
    public static void main(String[] args) {
        Enseignant e=new Enseignant();
    }
}
```



The screenshot shows a Java IDE window with tabs for Problems, Javadoc, Declaration, and Console. The Console tab is active, displaying the output of the program: "vous etes une personne", "Vous etes Salariée", and "Vous etes Enseignant". The window title is "<terminated> Enseignant [Java Application] C:\Program Files\Java\".

```
<terminated> Enseignant [Java Application] C:\Program Files\Java\
vous etes une personne
Vous etes Salariée
Vous etes Enseignant
```



Remarque: Héritage

L'héritage permet de créer une relation de type

« A est un B » ou « A est une sorte de B »

Pour créer une relation d'héritage on doit poser la question ?

Est-ce que A est une sorte de B ?

Est-ce que A est un B ?

Exemple:

Est-ce que Etudiant est une Personne ?

Est-ce que Salariée est une Personne ?

Est-ce que Enseignant est une Salariée ?

Est-ce que Chef_département est une Salariée ?

Remarque: Héritage

Supposons que nous disposons de la classe Point et nous souhaitons créer une classe PointCol afin de manipuler des points colorés dans un plan.

```
class Point {  
    private int x;  
    private int y;  
  
    public Point ()  
    {  
    }  
  
    public Point (int absc, int ord)  
    {x = absc;  
     y = ord;  
    }  
  
    public void déplacer (int dx, int dy)  
    {x = x + dx;  
     y = y + dy;  
    }  
  
    public void afficher()  
    {  
        System.out.println("je suis un point de coordonnées:"+x+" "+y);  
    }  
}
```

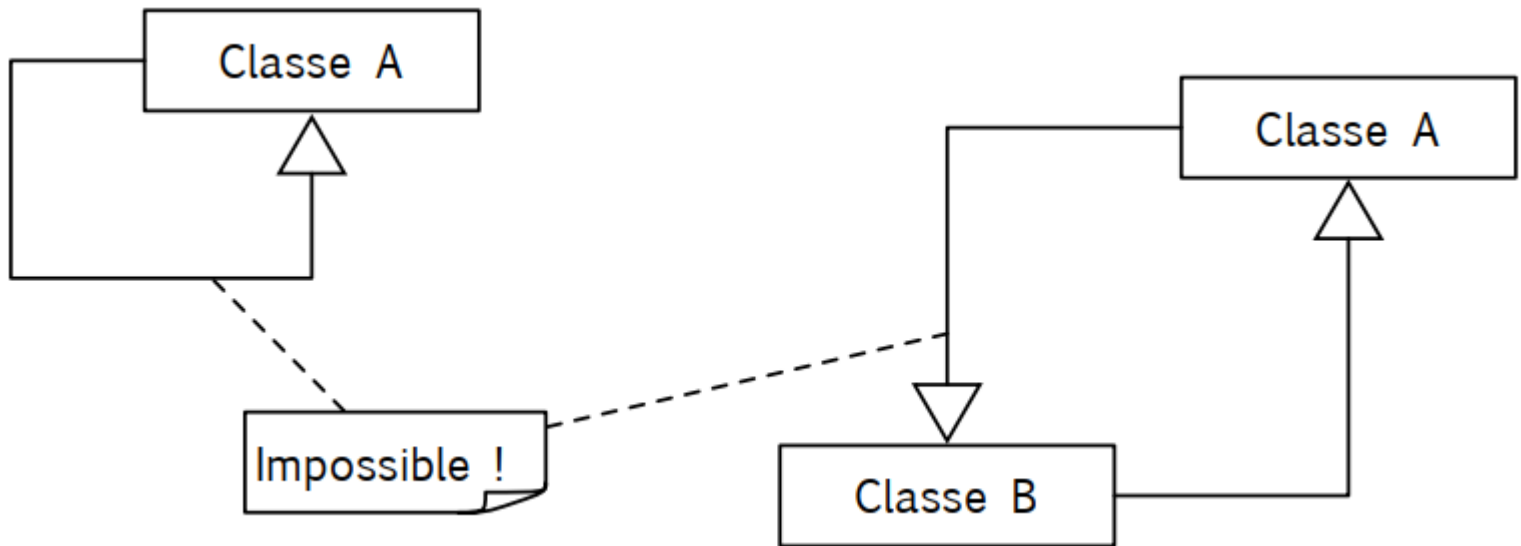
Héritage

Supposons que nous disposons de la classe `Point` et nous souhaitons créer une classe **`PointCol`** afin de manipuler des points colorés dans un plan.

La classe **`PointCol`** peut disposer des mêmes attributs et fonctionnalités que la classe `Point` auxquels on ajoute un attribut couleur représentant la couleur d'un point et une méthode `colorer` permettant d'attribuer une couleur à un point.

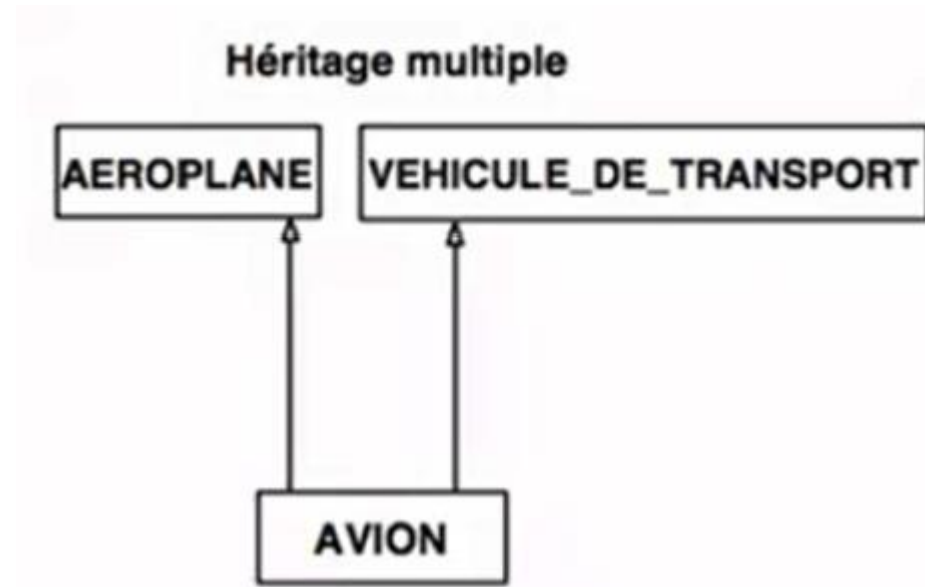
Remarque: Héritage

- **Relation non-réflexive, non-symétrique !**



Héritage multiple

- Une classe peut avoir plusieurs classes parents. On parle alors d'héritage multiple.



- Incompatible avec java
- langage C++ est un des langages objet permettant son implantation effective.

La redéfinition (**overriding**)

La redéfinition (**overriding**) signifie avoir deux méthodes avec le même nom et les mêmes paramètres, l'une des méthodes est dans la classe parente et l'autre dans la classe fille.

La redéfinition permet à une classe fille de fournir une implémentation spécifique d'une méthode déjà fournie à sa classe parente.

Exemple 1: La redéfinition (**overriding**)

```
public class Animal {  
    public void seDeplacer() {  
        System.out.println("Un animal peut se déplacer");  
    }  
}  
  
public class Chien extends Animal{  
    public void seDeplacer() {  
        System.out.println("Un chien peut courir");  
    }  
}  
  
public class Oiseaux extends Animal{  
    public void seDeplacer() {  
        System.out.println("Un oiseau peut voler");  
    }  
}  
  
public class Serpent extends Animal{  
    public void seDeplacer() {  
        System.out.println("Un serpent peut ramper");  
    }  
}
```

Exemple 1: La redéfinition (**overriding**)

```
public class Execution {
```

```
    void test(Animal c) {  
        c.seDeplacer();  
    }
```

```
    public static void main(String[] args) {  
        Execution p=new Execution();  
        Animal b=new Animal();  
        p.test(b);  
    }
```

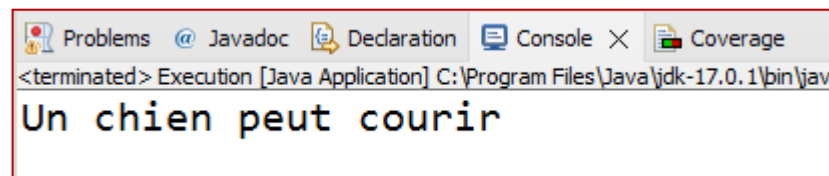
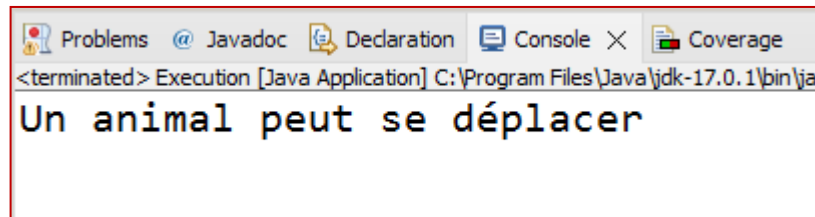
```
}
```

```
public class Execution {
```

```
    void test(Animal c) {  
        c.seDeplacer();  
    }
```

```
    public static void main(String[] args) {  
        Execution p=new Execution();  
        Chien b=new Chien();  
        p.test(b);  
    }
```

```
}
```



Exemple 1: La redéfinition (**overriding**)

```
public class Execution {
```

```
    void test(Animal c) {  
        c.seDeplacer();  
    }
```

```
    public static void main(String[] args) {  
        Execution p=new Execution();  
        Animal b=new Animal();  
        p.test(b);  
    }
```

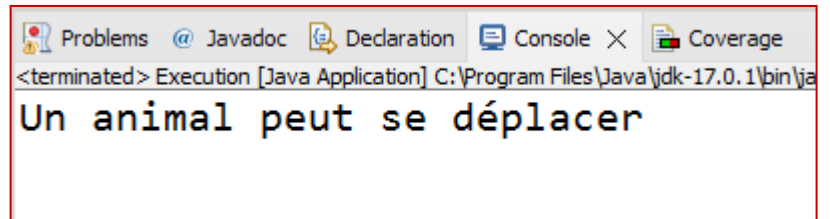
```
}
```

```
public class Execution {
```

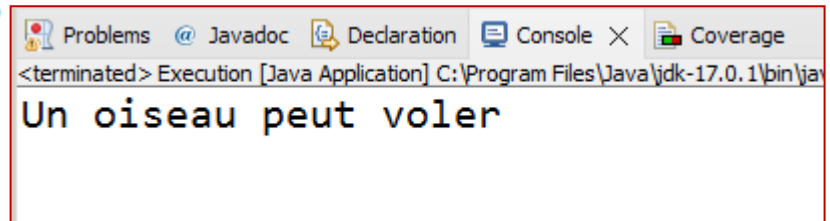
```
    void test(Animal c) {  
        c.seDeplacer();  
    }
```

```
    public static void main(String[] args) {  
        Execution p=new Execution();  
        Oiseaux b=new Oiseaux();  
        p.test(b);  
    }
```

```
}
```



Problems Javadoc Declaration Console X Coverage
<terminated> Execution [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\jav
Un animal peut se déplacer



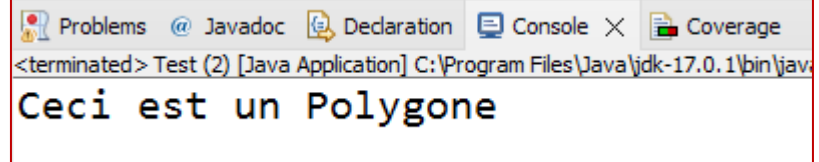
Problems Javadoc Declaration Console X Coverage
<terminated> Execution [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\jav
Un oiseau peut voler

Exemple 2: La redéfinition (**overriding**)

```
public class Polygone {  
    void afficher() {  
        System.out.println("Ceci est un Polygone");  
    }  
}  
  
public class Rectangle extends Polygone {  
    void afficher() {  
        System.out.println("Ceci est un Rectangle");  
    }  
}  
  
public class Triangle extends Polygone {  
    void afficher() {  
        System.out.println("Ceci est un Triangle");  
    }  
}
```

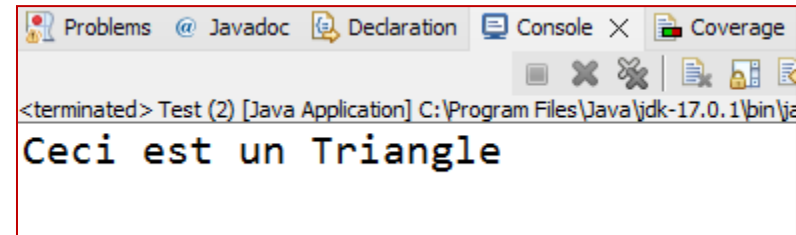
Exemple 2: La redéfinition (**overriding**)

```
public class Test {  
    void shapetype(Polygone A) {  
        A.afficher();  
    }  
    public static void main(String[] args) {  
        Test b=new Test();  
        Polygone v=new Polygone();  
        b.shapetype(v);  
    }  
}
```



Problems @ Javadoc Declaration Console X Coverage
<terminated> Test (2) [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\jav
Ceci est un Polygone

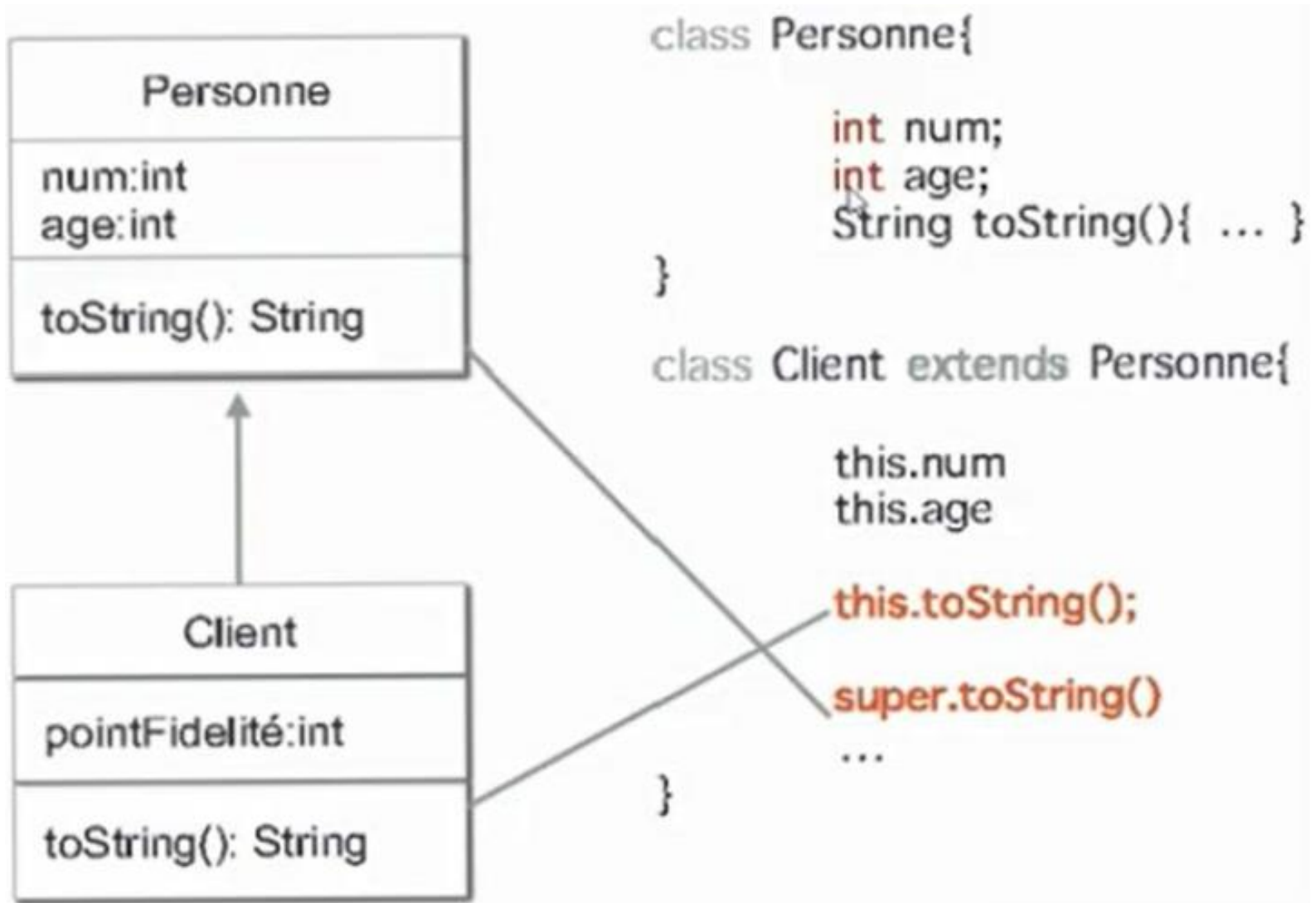
```
public class Test {  
    void shapetype(Polygone A) {  
        A.afficher();  
    }  
    public static void main(String[] args) {  
        Test b=new Test();  
        Triangle v=new Triangle();  
        b.shapetype(v);  
    }  
}
```



Problems @ Javadoc Declaration Console X Coverage
<terminated> Test (2) [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\ja
Ceci est un Triangle

La redéfinition (**overriding**)

Utilisation de **Super** et **this**



La redéfinition (**overriding**)

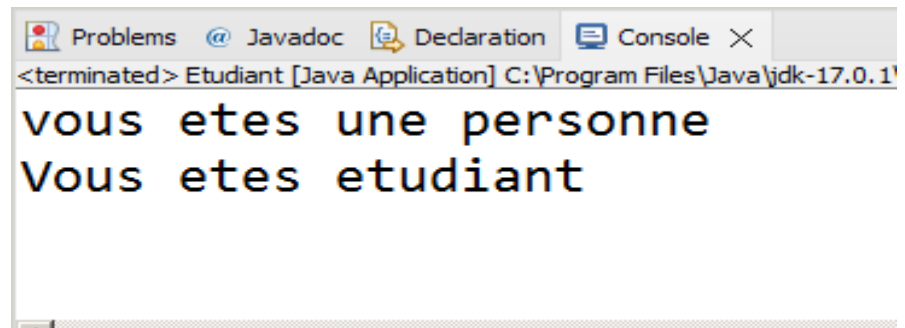
Utilisation de **Super** et **this**

```
public class Personne {  
    String nom;  
    String prenom;  
    int age;  
    void affichage() {  
        System.out.println("vous etes une personne");  
    }  
    public static void main(String[] args) {  
  
    }  
}
```

La redéfinition (**overriding**)

Utilisation de **Super** et **this**

```
public class Etudiant extends Personne{
    int matricule;
    void affichage() {
        System.out.println("Vous etes etudiant");
    }
    void display() {
        super.affichage();
        this.affichage();
    }
    public static void main(String[] args) {
        Etudiant e=new Etudiant();
        e.display();
    }
}
```



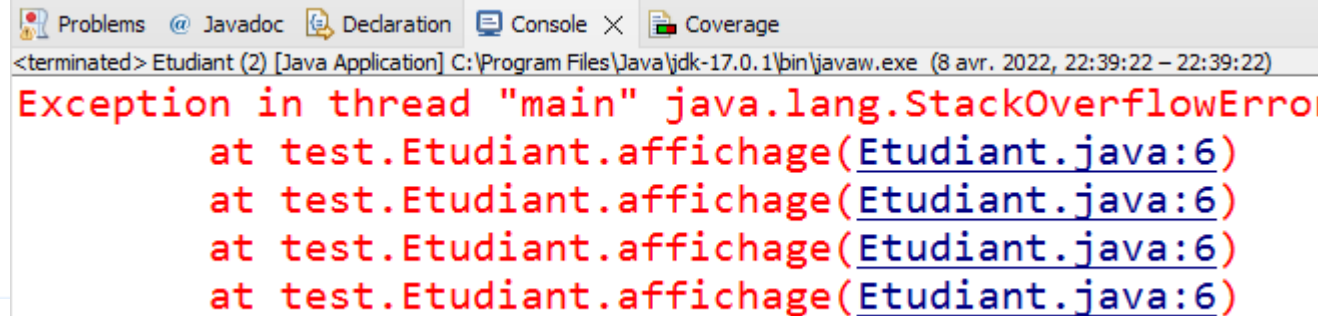
La redéfinition (**overriding**)

Utilisation de **Super** et **this**

super sert à accéder les définitions de classe au niveau de la classe **parente** de la classe considérée

this sert à accéder à la classe courante.

```
public class Personne {  
    String nom;  
    String Prenom;  
    int age;  
    void affichage() {  
        System.out.println("Vous etes une personne");  
    }  
    public static void main(String[] args) {
```



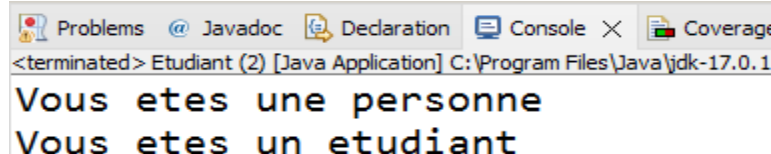
The screenshot shows an IDE window with tabs for Problems, Javadoc, Declaration, Console, and Coverage. The Console tab is active, displaying the following error message:

```
<terminated> Etudiant (2) [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (8 avr. 2022, 22:39:22 - 22:39:22)  
Exception in thread "main" java.lang.StackOverflowError  
    at test.Etudiant.affichage(Etudiant.java:6)  
    at test.Etudiant.affichage(Etudiant.java:6)  
    at test.Etudiant.affichage(Etudiant.java:6)  
    at test.Etudiant.affichage(Etudiant.java:6)
```

```
    }  
}  
  
public class Etudiant extends Personne{  
    int matricule;  
    void affichage() {  
        affichage();  
        System.out.println("Vous etes un etudiant");  
    }  
    public static void main(String[] args) {  
        Etudiant p=new Etudiant();  
        p.affichage();  
    }  
}
```

```
public class Personne {  
    String nom;  
    String Prenom;  
    int age;  
    void affichage() {  
        System.out.println("Vous etes une personne");  
    }  
    public static void main(String[] args) {  
  
    }  
}
```

```
public class Etudiant extends Personne{  
    int matricule;  
    void affichage() {  
        super.affichage();  
        System.out.println("Vous etes un etudiant");  
    }  
    public static void main(String[] args) {  
        Etudiant p=new Etudiant();  
        p.affichage();  
  
    }  
}
```



<terminated> Etudiant (2) [Java Application] C:\Program Files\Java\jdk-17.0.1
Vous etes une personne
Vous etes un etudiant

Différence entre surcharge et redéfinition

La **surcharge (overloading)** survient lorsque deux méthodes ou plus dans une classe ont le même nom de méthode mais avec des paramètres différents.

La **redéfinition (overriding)** signifie avoir deux méthodes avec le même nom et les mêmes paramètres, l'une des méthodes est dans la classe parente et l'autre dans la classe fille.

Règles sur l'utilisation de la surcharge et La redéfinition

- La **redéfinition** d'une méthode héritée doit impérativement conserver la déclaration de la méthode de base (type et nombre de paramètres et la valeur de retour doivent être identiques) et fournir dans une sous-classe une nouvelle implémentation de la méthode.
- Si la signature de la méthode change, ce n'est plus une redéfinition mais **une surcharge** (surdéfinition).

La surcharge + La redéfinition = polymorphisme



*Class1.java × Class2.java

```
1 package test;
2 public class Class1 {
3     // Constructeur par défaut
4     Class1(){
5         System.out.println("Constructeur de la Class 1");
6     }
7     // Constructeur avec parametres
8     Class1(int val){
9
10        System.out.println("Constructeur avec parametres Class 1 val:"+val);
11    }
12    public static void main(String[] args) {
13        Class1 p=new Class1();
14    }
15 }
16
17
```

Problems @ Javadoc Declaration Console × Coverage

<terminated> Class1 [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (9 avr. 2022, 01:44:23 - 01:44:23)

Constructeur de la Class 1



Class1.java × Class2.java

```
1 package test;
2 public class Class1 {
3     // Constructeur par défaut
4     Class1(){
5         System.out.println("Constructeur de la Class 1");
6     }
7     // Constructeur avec parametres
8     Class1(int val){
9
10        System.out.println("Constructeur avec parametres Class 1 val:"+val);
11    }
12    public static void main(String[] args) {
13        Class1 p=new Class1(11);
14    }
15 }
16
17
```

Problems @ Javadoc Declaration Console × Coverage

<terminated> Class1 [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (9 avr. 2022, 01:47:24 – 01:47:25)

Constructeur avec parametres Class 1 val:11

Writable

Smart Insert

16 : 1 : 357





Class1.java × Class2.java

```
1 package test;
2 public class Class1 {
3     // Constructeur par défaut
4     Class1(){
5         System.out.println("Constructeur de la Class 1");
6     }
7     // Constructeur avec parametres
8     Class1(int val){
9         this();
10        System.out.println("Constructeur avec parametres Class 1 val:"+val);
11    }
12    public static void main(String[] args) {
13        Class1 p=new Class1(11);
14    }
15 }
16
```

Problems @ Javadoc Declaration Console × Coverage

<terminated> Class1 [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (9 avr. 2022, 01:54:04 – 01:54:05)

Constructeur de la Class 1

Constructeur avec parametres Class 1 val:11

Writable

Smart Insert

17 : 1 : 369





Class1.java Class2.java X

```
1 package test;
2
3 public class Class2 extends Class1{
4     Class2(){
5
6         System.out.println("Constructeur de la Class 2 (par défaut)");
7     }
8     Class2(int val){
9         System.out.println("Constructeur de la Class 2 (avec parametres) val"+val);
10    }
11    public static void main(String[] args) {
12        Class2 d=new Class2();
13    }
14 }
15
```

Problems Javadoc Declaration Console X Coverage

<terminated> Class2 [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (9 avr. 2022, 01:58:48 - 01:58:49)

```
Constructeur de la Class 1
Constructeur de la Class 2 (par défaut)
```

Writable

Smart Insert

17 : 1 : 324



Class1.java Class2.java ×

```
1 package test;
2
3 public class Class2 extends Class1{
4     Class2(){
5         super();
6         System.out.println("Constructeur de la Class 2 (par défaut)");
7     }
8     Class2(int val){
9         System.out.println("Constructeur de la Class 2 (avec parametres) val"+val);
10    }
11    public static void main(String[] args) {
12        Class2 d=new Class2();
13    }
14 }
15
16
```

Problems @ Javadoc Declaration Console × Coverage

<terminated> Class2 [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (9 avr. 2022, 02:01:02 – 02:01:03)

Constructeur de la Class 1

Constructeur de la Class 2 (par défaut)

Writable

Smart Insert

17 : 1 : 332





Class1.java Class2.java ×

```
1 package test;
2
3 public class Class2 extends Class1{
4     Class2(){
5         super(3);
6         System.out.println("Constructeur de la Class 2 (par défaut)");
7     }
8     Class2(int val){
9         System.out.println("Constructeur de la Class 2 (avec parametres) val"+val);
10    }
11    public static void main(String[] args) {
12        Class2 d=new Class2();
13    }
14 }
15
16
```

Problems @ Javadoc Declaration Console × Coverage

<terminated> Class2 [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (9 avr. 2022, 02:02:53 – 02:02:53)

Constructeur de la Class 1

Constructeur avec parametres Class 1 val:3

Constructeur de la Class 2 (par défaut)

Règles sur l'utilisation des constructeurs de la classe mère

Règle 1 : si vous invoquez **super(...)**, cela signifie que le constructeur en cours d'exécution passe la main au constructeur de la classe parente pour commencer à initialiser les attributs définis dans cette dernière. Ensuite il continuera son exécution.

Règle 2 : un appel de constructeur de la classe mère peut uniquement se faire qu'en première instruction d'une définition de constructeur.

Règle 3 : si la première instruction d'un constructeur ne commence pas par le mot clé **super** le constructeur par défaut de la classe mère est appelé.

Règle 4 : si vous invoquez **this()**, le constructeur considéré passe la main à un autre constructeur de la classe considérée.

Empêcher l'héritage avec le mot clé **final**

```
public final class Class1 {  
    // Constructeur par défaut  
    Class1(){  
        System.out.println("Constructeur de la Class 1");  
    }  
    // Constructeur avec parametres  
    Class1(int val){  
        this();  
        System.out.println("Constructeur avec parametres Class 1 val:"+val);  
    }  
    public static void main(String[] args) {  
        Class1 p=new Class1(11);  
    }  
}
```

```
public class Class2 extends Class1{
```

X Erreur

Exercice:

Trouvez et corriger les sept (07) erreurs pour que ce programme fonctionne

```
package a;  
class A {  
    private int x;  
    public A(int px){  
        px=x ;  
    }  
}
```

```
package a;  
class B extends A {  
    private int y;  
    public B(int px, int py){  
        x=px ;  
        y=py;  
    }  
    public int somme (){  
        return x+y;  
    }  
}
```

```
package b;  
class Test{  
    public static void main (String [] args)  
        B b=new B(8,7);  
        System.out.print(somme());  
    }  
}
```

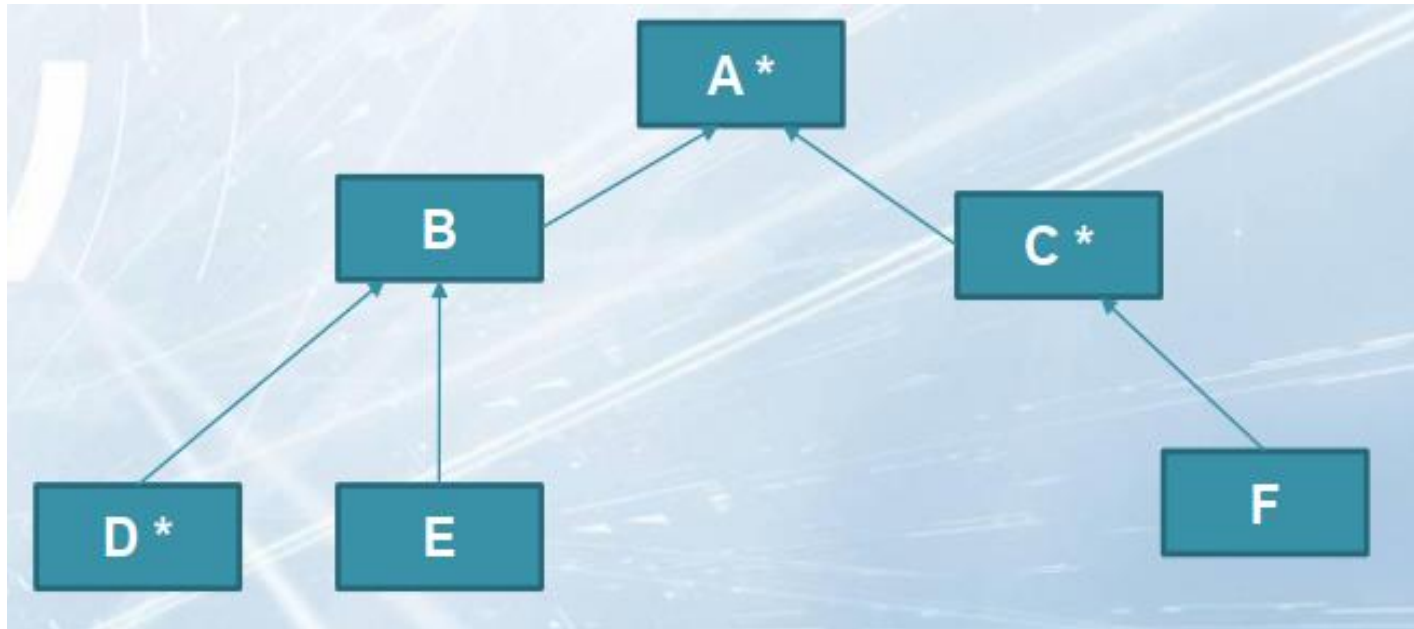
Solution :

```
package a;
class A {
    private int x;
    public A(int px){
        x=px; // au lieu de px=x
    }
    int getX(){
        return x;
    }
}
```

```
package a;
public class B extends A {
    private int y;
    public B(int px, int py){
        super(px);
        y=py;}
    public int somme (){
        return getX()+y //au lieu de x+y;
    }
}
```

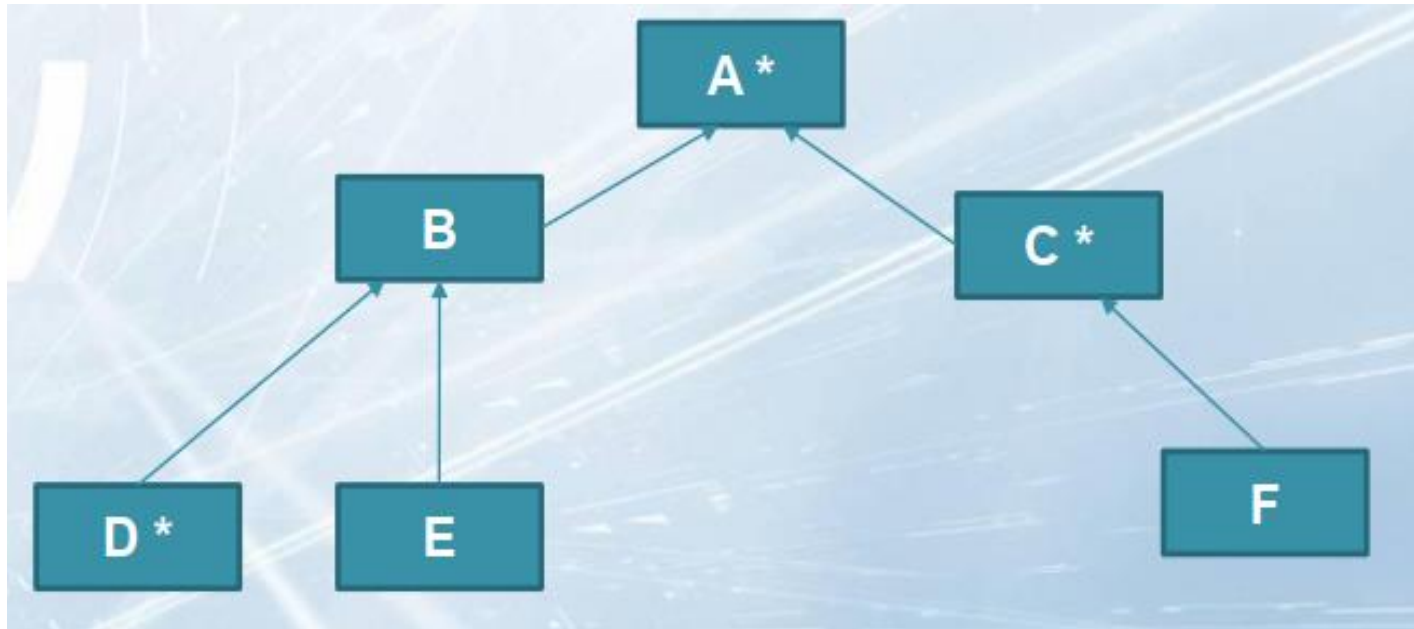
```
package b;
import a.B; (1)
class Test{
    public static void main (String []
args){
        B b=new B(8,7);
        System.out.print(b.somme());
    }
}
```

Soit une méthode $F()$ définie dans A et redéfinie dans les classes C et D



Appel de f dans A conduit à l'appel de f de ...
Appel de f dans B conduit à l'appel de f de ...
Appel de f dans C conduit à l'appel de f de ...
Appel de f dans D conduit à l'appel de f de ...
Appel de f dans E conduit à l'appel de f de ...
Appel de f dans F conduit à l'appel de f de ...

Soit une méthode f définie dans A et redéfinie dans les classes C et D



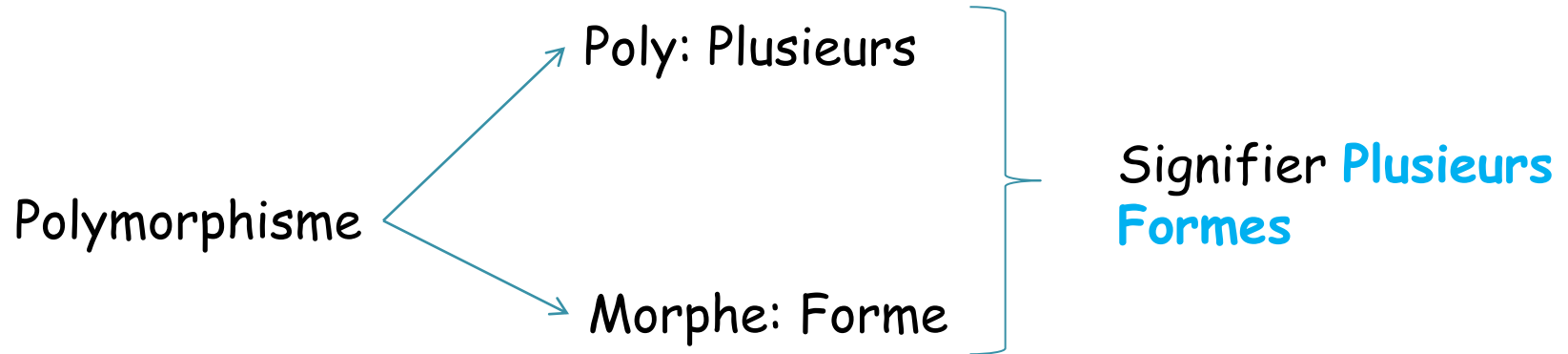
Appel de f dans A conduit à l'appel de f de ... A
Appel de f dans B conduit à l'appel de f de ... A
Appel de f dans C conduit à l'appel de f de ... C
Appel de f dans D conduit à l'appel de f de ... D
Appel de f dans E conduit à l'appel de f de ... A
Appel de f dans F conduit à l'appel de f de ... C



Polymorphisme

Polymorphisme

c'est quoi **Polymorphisme** ?

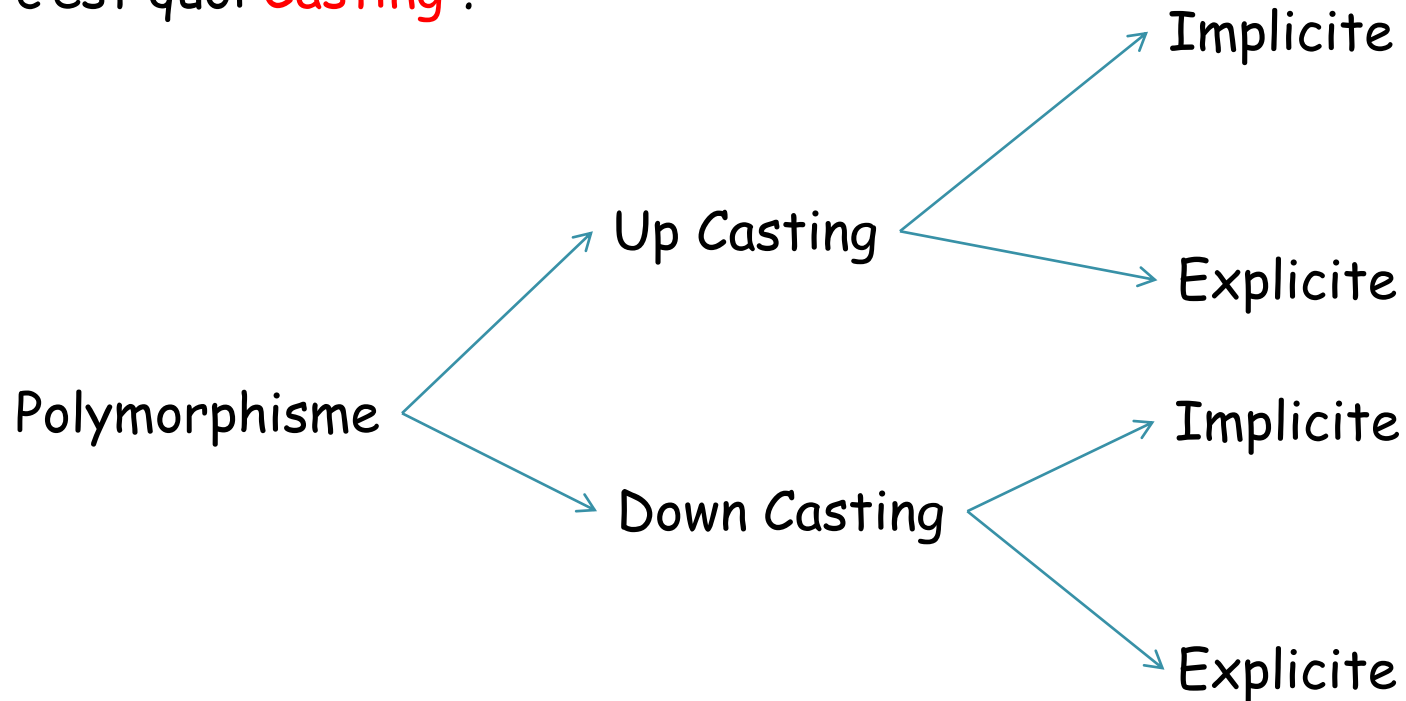


Un objet peut prendre plusieurs formes

Polymorphisme

c'est quoi **Casting** ?

Casting



Casting c'est l'attribution des rôles

Polymorphisme

Casting

Règles de transtypage ou de Casting (UpCasting + DownCasting)

Dans une instruction d'affectation (`refVar1 = refVar2;`),
le compilateur vérifie que :

le type de la variable `refVar1` est le même que celui de `refVar2`
ou un type parent du type de la variable `refVar2`.

"UpCasting" implicite/explicite : permet de convertir le type
d'une référence vers un type parent.

Polymorphisme

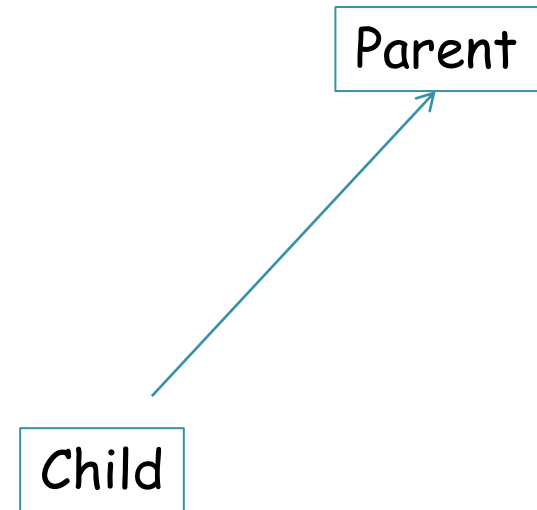
Exemple:

```
public class Parent {  
    String name;  
  
    void mtd(){  
        System.out.println("Parent");  
    }  
}
```

```
public class Child extends Parent{  
    int id;  
  
    void mtd() {  
        System.out.println("Child");  
    }  
}
```

```
public static void main(String[] args) {  
    Parent p=new Parent();  
    Child c=new Child();  
    p=c; //Child to Parent accepted implicit
```

UP Casting



Up casting: conversion sous classe vers Super classe est toujours Autorisé dans java explicite Or implicite

Polymorphisme

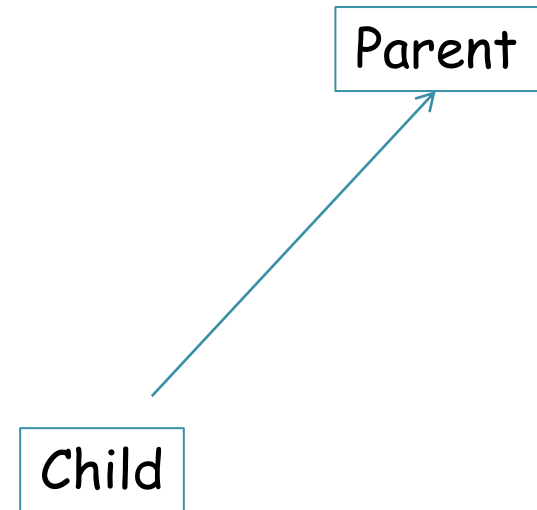
Exemple:

```
public class Parent {  
    String name;  
  
    void mtd(){  
        System.out.println("Parent");  
    }  
}
```

```
public class Child extends Parent{  
    int id;  
  
    void mtd() {  
        System.out.println("Child");  
    }  
}
```

```
public static void main(String[] args) {  
    Parent p=new Parent();  
    Child c=new Child();  
    p= (Parent) c; //Child to Parent accepted explicit
```

UP Casting



Up casting: conversion sous classe vers Super classe est toujours Autorisé dans java explicite Or implicite

Polymorphisme

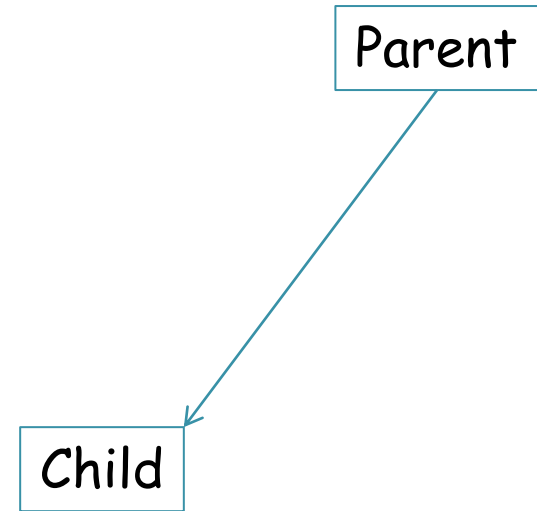
Exemple:

Down casting: conversion
Super classe vers sous classe

Down casting explicite:
interdit erreur de compilation

Down casting implicite:
autorisé par compilateur mais
JVM déclenchera une
exception

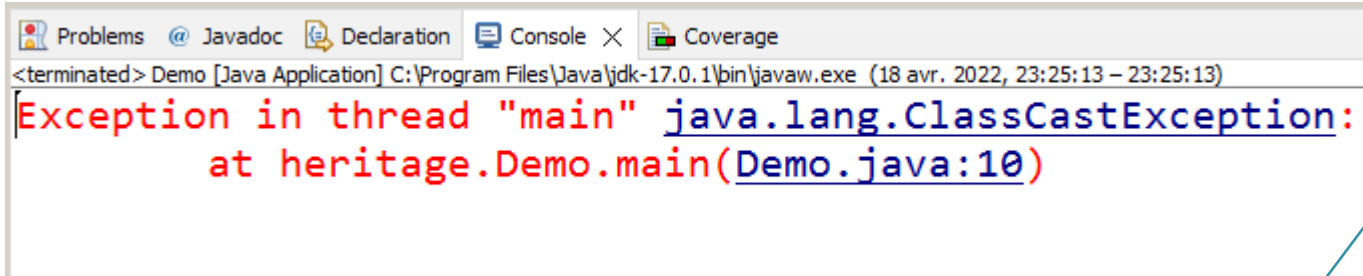
Down Casting



Polymorphisme

Exemple:

Down Casting



The screenshot shows an IDE console window with the following text:

```
<terminated> Demo [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (18 avr. 2022, 23:25:13 - 23:25:13)  
Exception in thread "main" java.lang.ClassCastException:  
    at heritage.Demo.main(Demo.java:10)
```

Parent

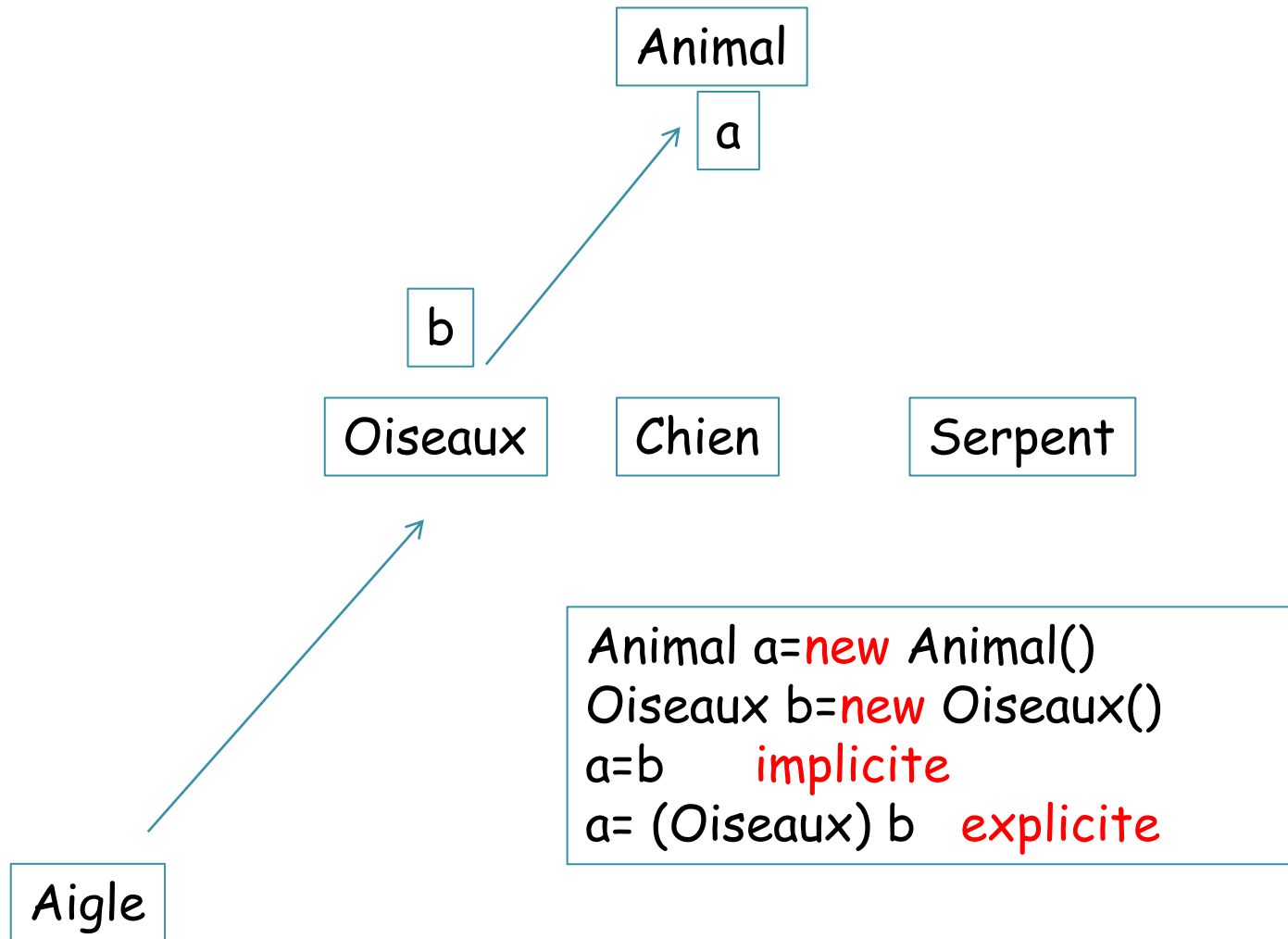
Child

```
public static void main(String[] args) {  
    Parent p=new Parent();  
    Child c=new Child();  
  
    c=p; //Parent to Child cannot be accepted by compiler implicit  
    c=(Child) p; //Parent to Child cannot be accepted by JVM explicit
```

Polymorphisme

Exemple:

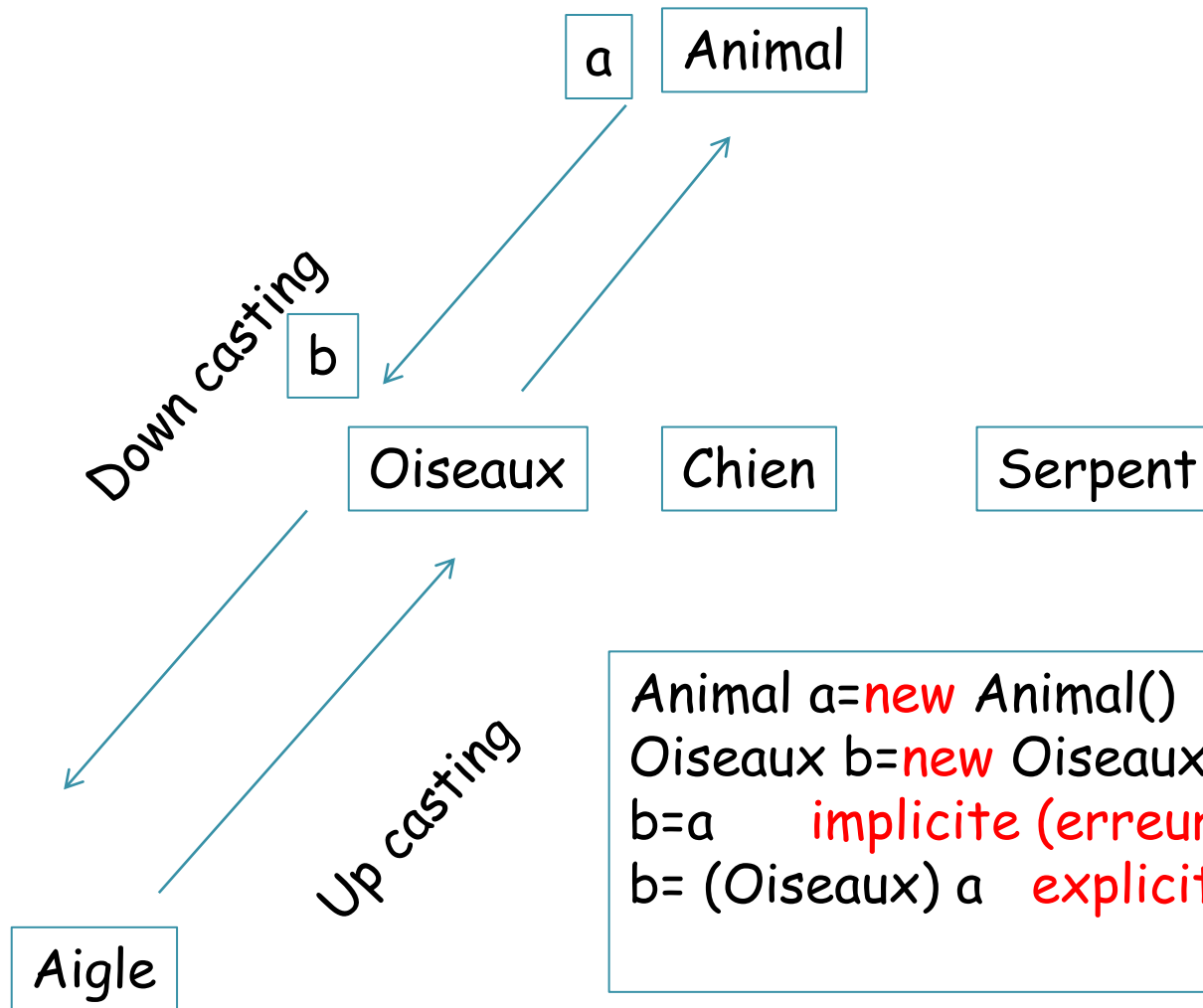
Down Casting



Polymorphisme

Exemple:

Down Casting



```
Animal a=new Animal()  
Oiseaux b=new Oiseaux()  
b=a      implicite (erreur)  
b= (Oiseaux) a  explicite (erreur)
```

le mot clé **final**

final

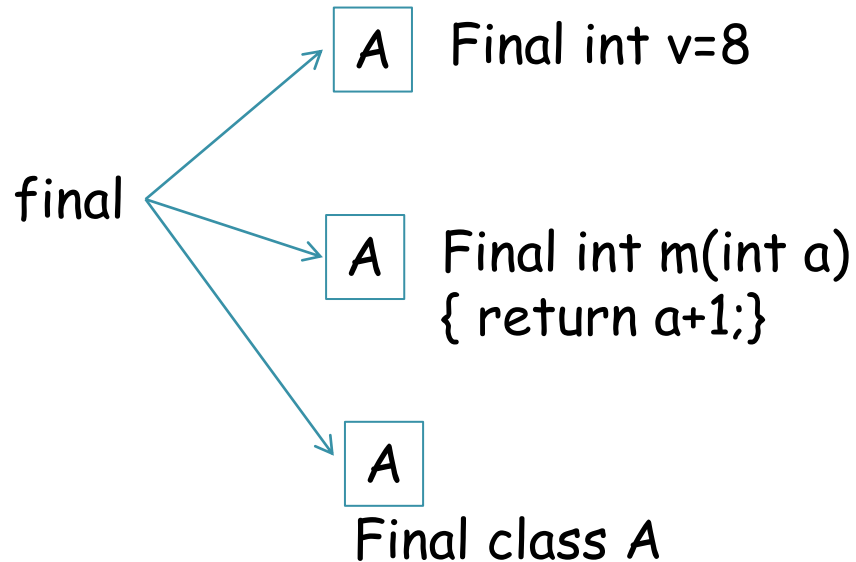
Attribut: on ne peut pas changer (constante)
dans cette classe ou dans les sous classes

Méthode: on ne peut pas redéfinir dans les
sous classes

Classe: on ne peut pas hériter

le mot clé **final**

Soit B une classe dérive de A



B B b=new B() X
b.v=10;

B int m(int b) X
{ return b+4;}

B class B extends A X

La class Objet

- Racine de l'arbre d'héritage des classes : `java.lang.Object` □
- Héritée par toutes les classes sans exception
- Object n'a pas de variable d'instance ni de variable de classe
- Object fournit plusieurs méthodes Couramment utilisées sont les méthodes `toString` et `equals`

La class Objet méthode `toString()` et `hashCode()`

`toString()` renvoie:

- Description de l'objet sous la forme d'une chaîne de caractères
- Nom de la classe, suivie de "@" et de la valeur de la méthode

`hashCode()`:

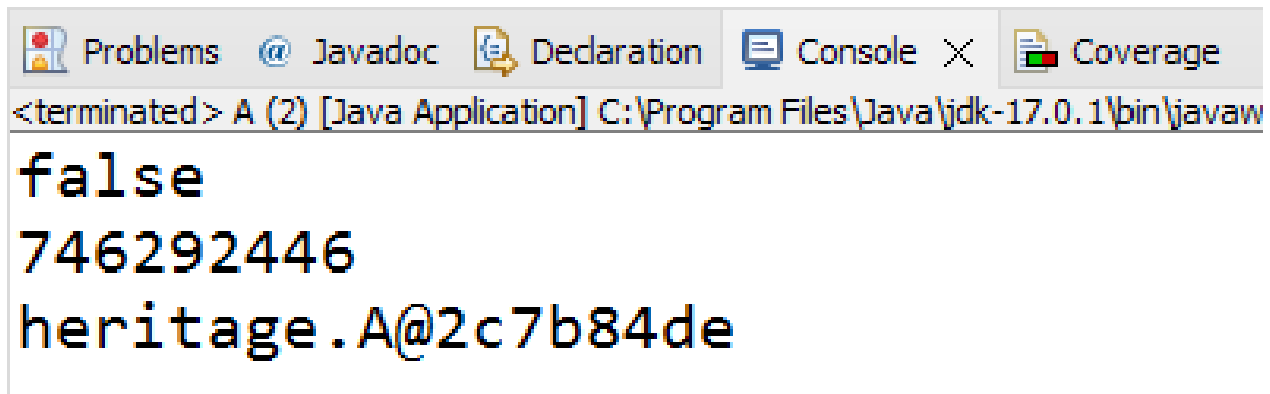
- `hashCode()` renvoie la valeur hexadécimale de l'adresse mémoire de l'objet

`equals()` : comparaison de l'objet courant avec un autre

La class Objet méthode `toString()` et `hashCode()`

Exemple:

```
public static void main(String[] args) {  
    A a=new A();  
    A b=new A();  
    System.out.println(a.equals(b));  
    System.out.println(a.hashCode());  
    System.out.println(a.toString());  
}
```



The screenshot shows a Java IDE interface with a console window. The console window has tabs for Problems, Javadoc, Declaration, Console, and Coverage. The Console tab is active, showing the output of the program. The output consists of three lines: `false`, `746292446`, and `heritage.A@2c7b84de`. The first line is the result of `a.equals(b)`, the second is the result of `a.hashCode()`, and the third is the result of `a.toString()`.

```
<terminated> A (2) [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw  
false  
746292446  
heritage.A@2c7b84de
```



Classes abstraites et Interfaces



Classes abstraites

Méthode abstraite

Il peut donc, dans certains cas, être utile de définir une méthode sans en donner le code. Utilisation du mot clé

`abstract`

Exemple:

```
abstract double surface();
```

au lieu de

```
double surface(){ return 0; }
```

Classe abstraite

Si une méthode est déclarée comme abstract la classe au quelle définit cette méthode doit forcément déclarer comme abstract mais **l'inverse est faux**.

Exemple:

```
public abstract class Parent {  
    String name;  
  
    abstract void mtd();  
  
}
```

Classe abstraite

Si une méthode est déclarée comme abstract la classe au quelle définit cette méthode doit forcément déclarer comme abstract mais **l'inverse est faux**.

Exemple:

```
public abstract class Parent {  
    String name;  
  
    abstract void mtd();  
  
}
```

Une classe abstraite pas forcément ses méthodes sont abstraits.

Classe abstraite

- Une classe déclare comme **abstraite** on ne peut pas créer des objets à partir de cette classe.
- Définit une classe comme **abstraite** nécessite des classes fille pour l'utiliser
- Les méthodes déclarés avec le mot clé « **abstract** » elles sont héritées par les classe filles mais elles nécessitent une redéfinitions.



Classes interfaces

Classe interface

- Une classe déclare comme **interface** est une cas particulier d'une classe **abstraite**

- Une interface est une classe 100% **abstraite**

C'est-à-dire:

- Toutes ses méthodes sont déclarés **public abstract**
- Toutes ses attributs sont déclarés **static final**
- Pour utiliser une interface on doit utiliser le mot clé
« **implements** » au lieu de « **extends** »

Remarque

- Soit une Classe 1 déclare comme interface avec des méthodes abstraites.
- Si une classe 2 implémente la classe 1 elle doit forcément redéfinit **toutes** ces méthodes
- Sinon si la classe 2 implémente (redéfinit) partiellement les méthodes de la Classe 1, cette dernier doit être déclarer comme abstract

Rappel

- Une Classe peut implémenté plusieurs interfaces.
- Une interface pas d'un constructeur
- Une classe n'hérite pas d'un interface elle implémente.